

Software Engineering

MODULE 1: The Product and the Process

CO 1: Understand the software development process and Estimate different types of resources for the given project

- 1.1 Software Development Life Cycle models: Waterfall, RAD, Spiral , Agile process
- 1.2 Understanding software process, Process metric, CMM (Capability Maturity Model)levels
- 1.3 Planning & Estimation: Product metrics estimations- LOC, FP(Functional Point), COCOMO(Constructive Cost Model) models
- 1.4 Project Management Activities: Planning , Scheduling & Tracking

Software

- Software:

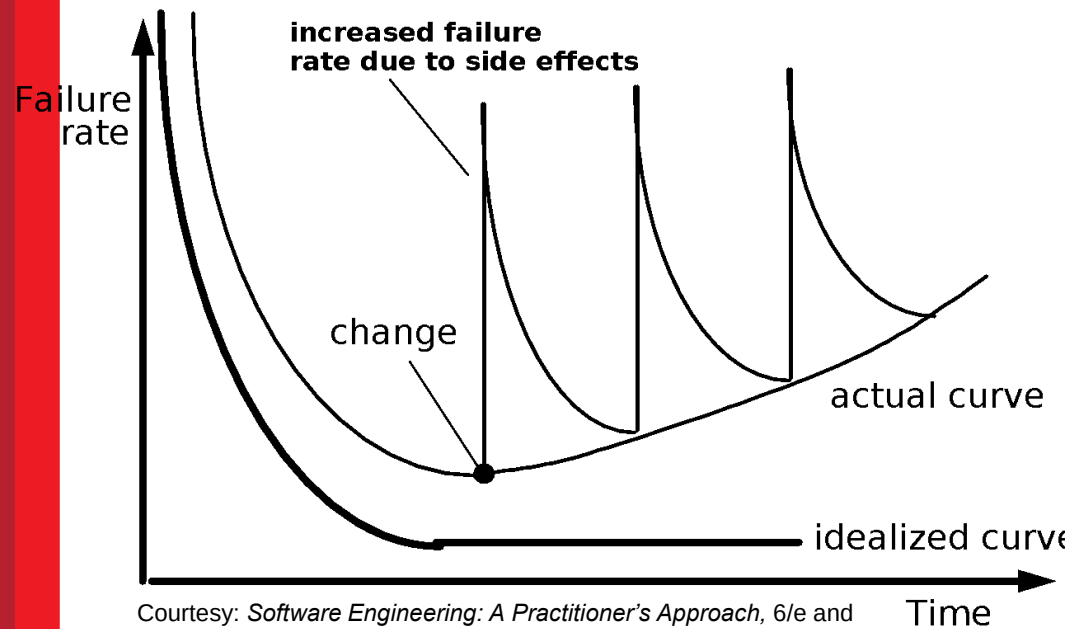
- Product: Supports or directly provides system functionality
- Produces, manages, acquires, modifies, displays, or transmits information
- Tool: Helps build other software Delivers computing potential

Software includes:

- Program/ Product :
 - Operating System
 - API/ Library Functions/ Apps
 - Databases
- Documents:
 - Problem Definition
 - Method to solve / approach
 - Code (Reference manual)
 - Users Manual
 - Test reports

Software

- Developed / Engineered
- Does not wear & tear



Courtesy: *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

- Complex

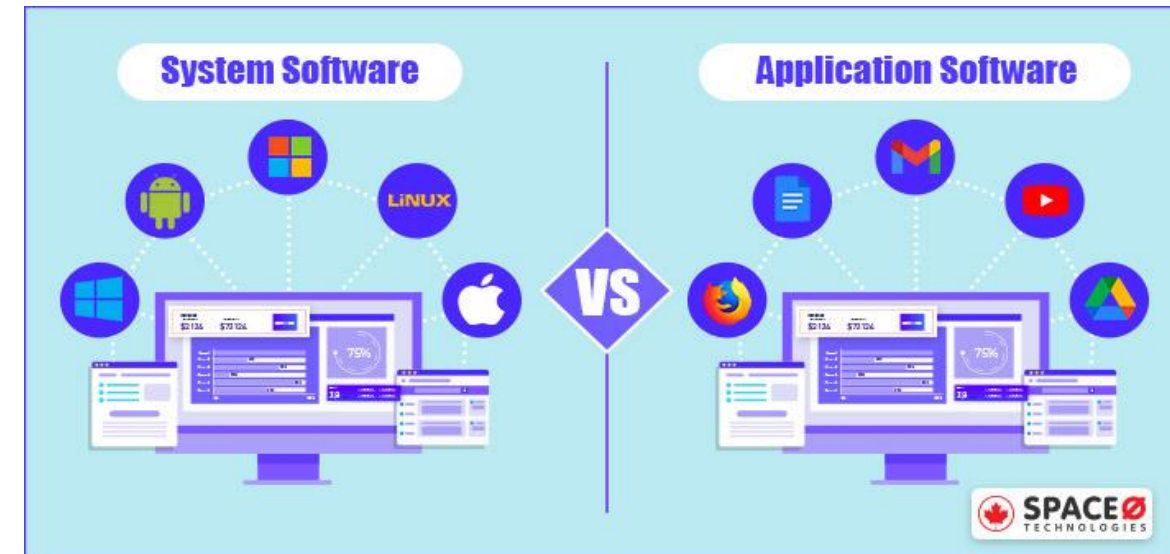


SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Types / Classification of Software

- **System Software** : A collection of programs written to service other programs
 - Operating System
 - Device Drivers
- **Application Software** : Stand alone program that solve a specific business need
 - Games
 - Apps
- **Generic Software**: Engineering or scientific software
- **Customized Software**: Embedded



Definition of Software Engineering

- **Software**
 - Document / Code
- **Engineering**
 - Systematic Development approach
 - Optimum utilization of resources
- **IEEE** defines **software engineering** as: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of **software**; that is, the application of **engineering** to **software**. (2) Study of approaches as in (1).
- Fritz Bauer defined Software engineering as the “*establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.*”

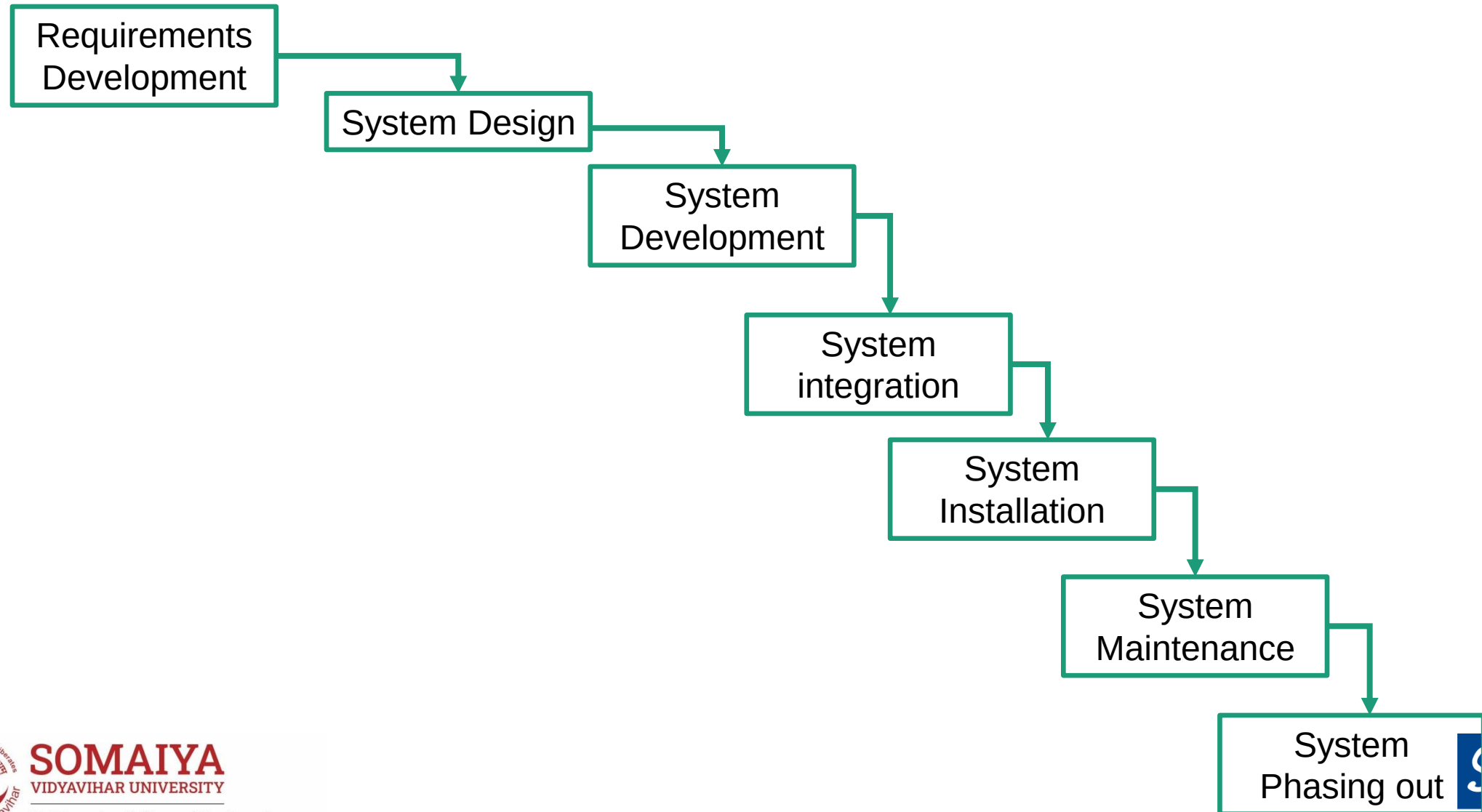
Definition of Software Engineering

- Software Engineering is the branch of engineering which deals with the establishment and use of sound engineering principles to obtain economical Software that is reliable and works efficiently on real machines.

System Engineering & Software Engineering

- System: Set of parts interconnected in such a way to achieve common objectives
- System Engineering focuses on a specific domain such as business or product
- Software: Program, Installation manual, Documentation

Processes of System Engineering



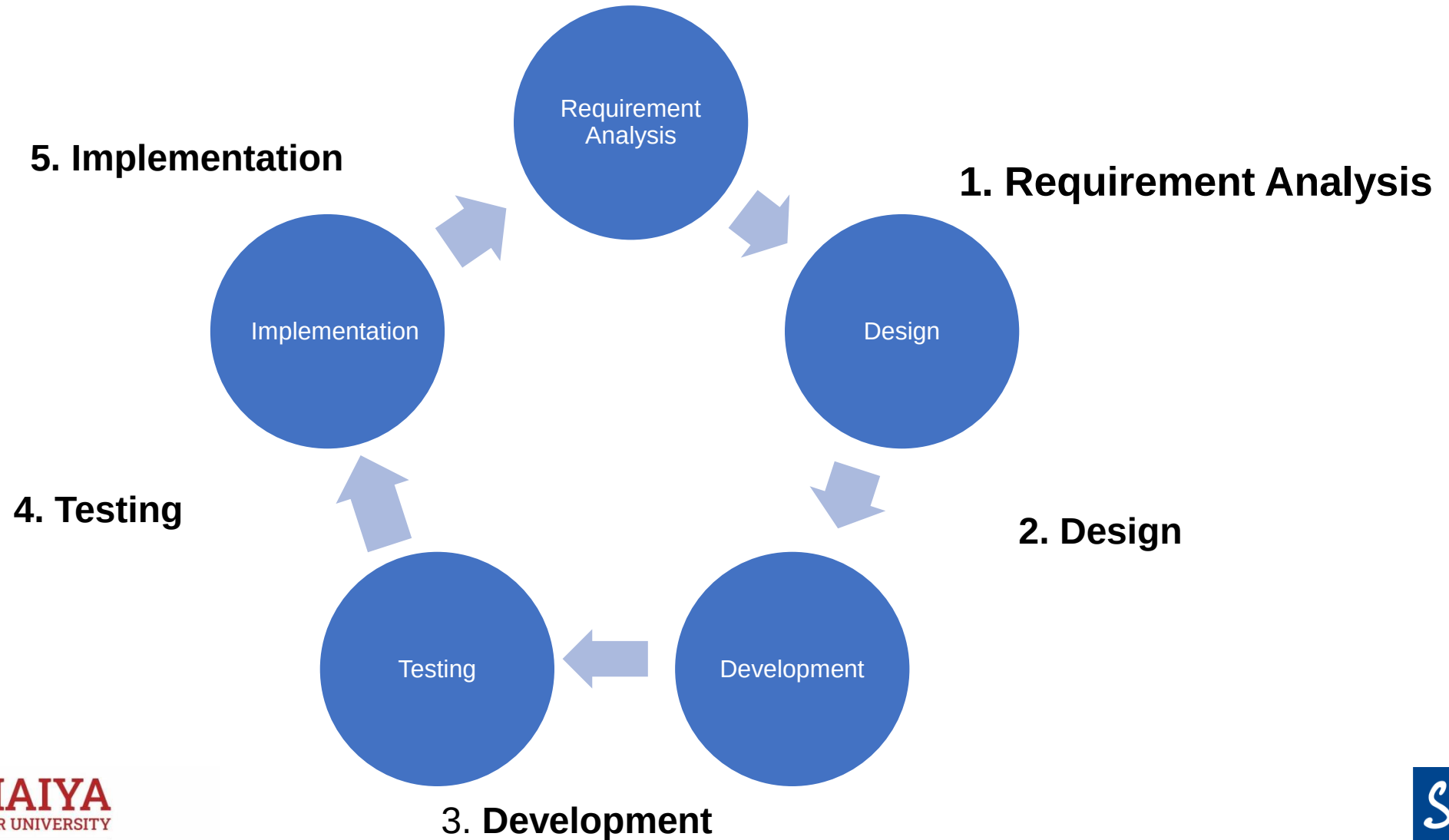
SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

4 Ps Related to Software Engineering (SE)

- Product
 - The deliverables
- Process
 - Development
 - Testing
- Project
 - Systematic approach with specific characteristics
- People
 - Designers, Analysts, Developers, Users

Activities performed in life cycle of a Software Development projects



Steps involved in Software Development

1. Identification of Problem
2. Problem Definition
3. Exploring approach(s) to solve the problem
4. Penning down/ finalizing **the** approach; listing the steps
5. Development
6. Testing
7. Deployment
8. Maintenance

These steps are iterative and some of them run parallel

Generic Software Development Process Models

1. Code & Fix Model
2. Waterfall model
3. Rapid Application Development (RAD)
4. Spiral
5. Iterative
6. Evolutionary
7. Agile

Code & Fix Model

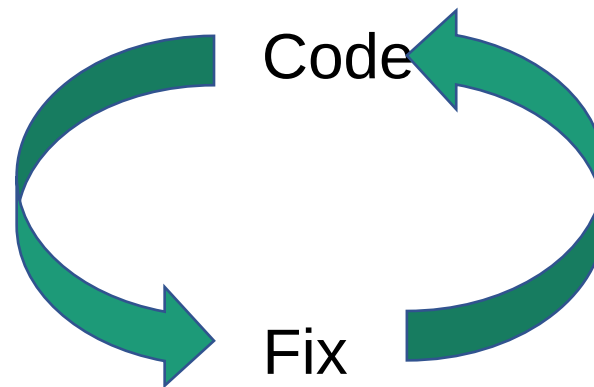
Code the software & Fix Modify (similar to trail & error)

Generally used for SMALL projects

Very short delivery time

SE is restricted only to management of program

NO way to keep track of of requirements, business logic and design



What / who / why is Process Models?

- **What:** Go through a series of predictable steps--- a road map that helps you create a timely, high-quality results.
- **Who:** Software engineers and their managers, clients also. People adapt the process to their needs and follow it.
- **Why:** Provides stability, control, and organization to an activity that can if left uncontrolled, become quite chaotic. However, modern software engineering approaches must be agile and demand ONLY those activities, controls and work products that are appropriate.
- **What Work products:** Programs, documents, and data
- **What are the steps:** The process you adopt depends on the software that you are building.
- **How to ensure right:** A number of software process assessment mechanisms that enable us to determine the maturity of the software process. However, the quality, timeliness and long-term viability of the software are the best indicators of the efficacy of the process you use.

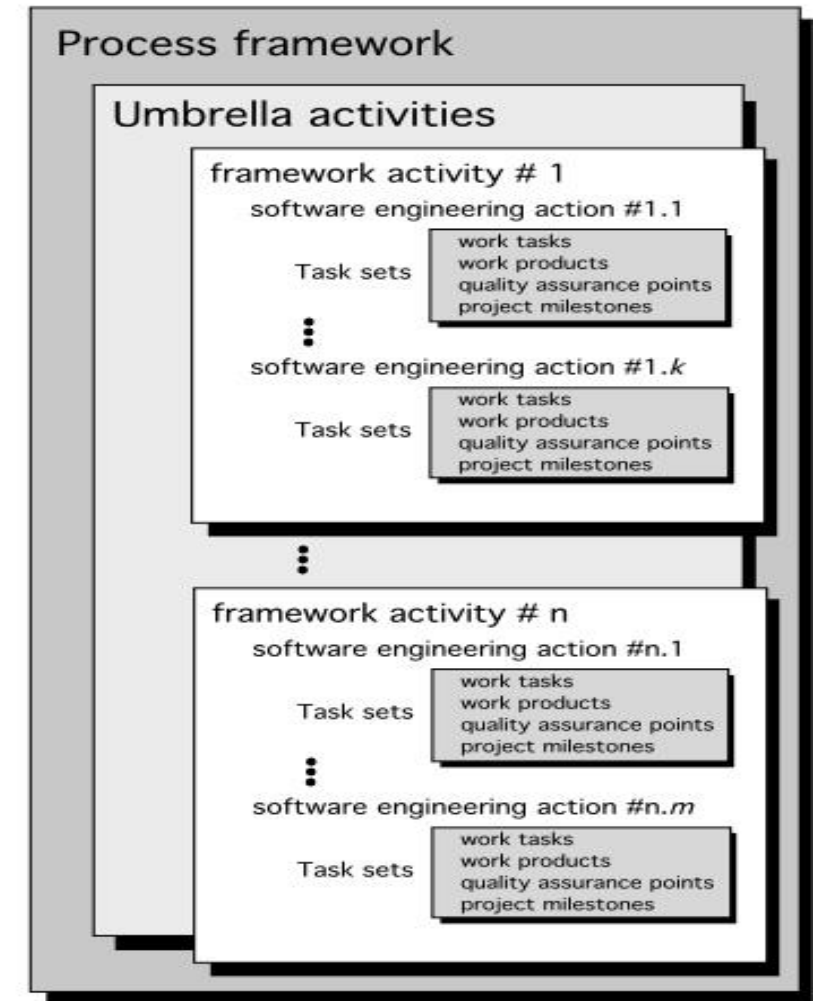
Definition of Software Process

- A **framework** for the activities, actions, and tasks that are required to build high-quality software.
- SP defines the approach that is taken as software is engineered.

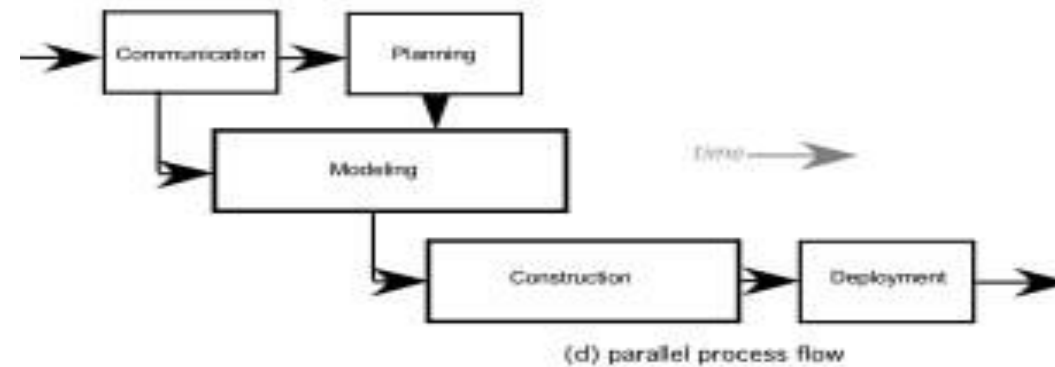
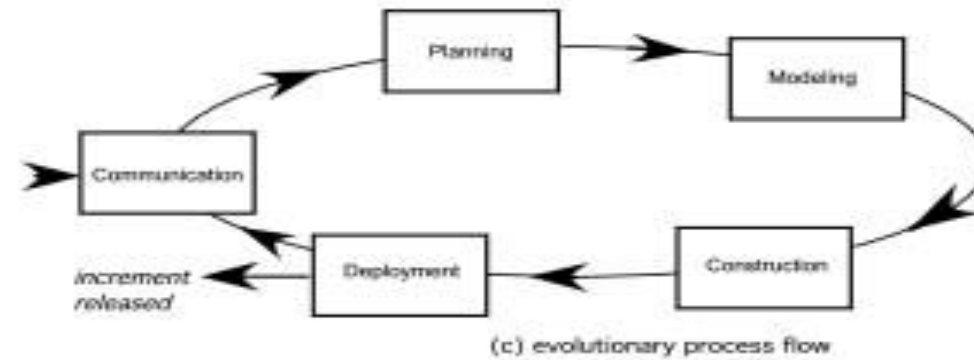
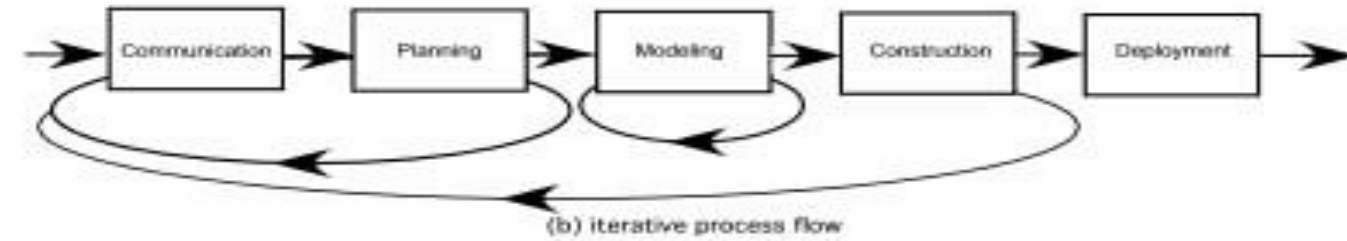
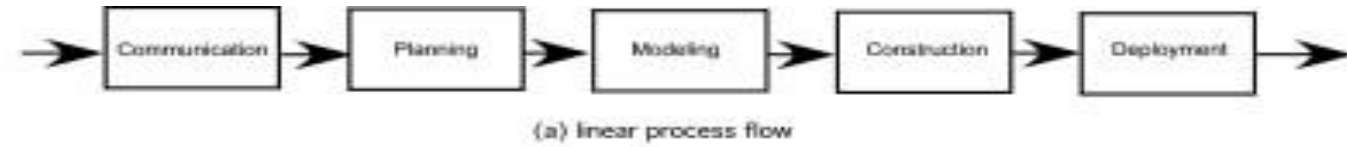
A Generic Process Model

- A generic process framework for software engineering defines five framework activities- communication, planning, modeling, construction, and deployment.
- In addition, a set of umbrella activities- project tracking and control, risk management, quality assurance, configuration management, technical reviews, and others are applied throughout the process.
- Next question is: how the framework activities and the actions and tasks that occur within each activity are organized with respect to sequence and time? See the **process flow** for answer.

Software process



Process Flow



Linear process flow

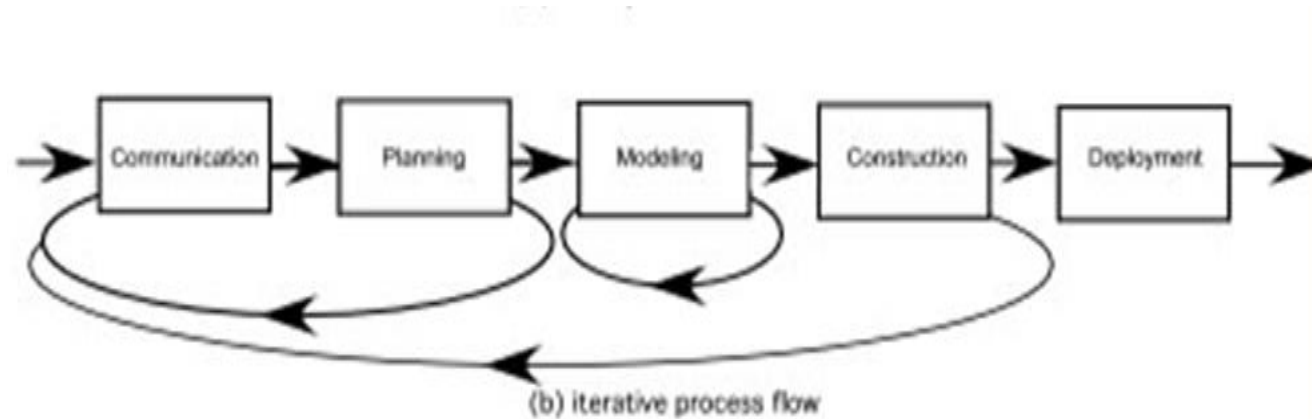
- Linear process flow executes each of the five activities in sequence.



(a) Linear process flow

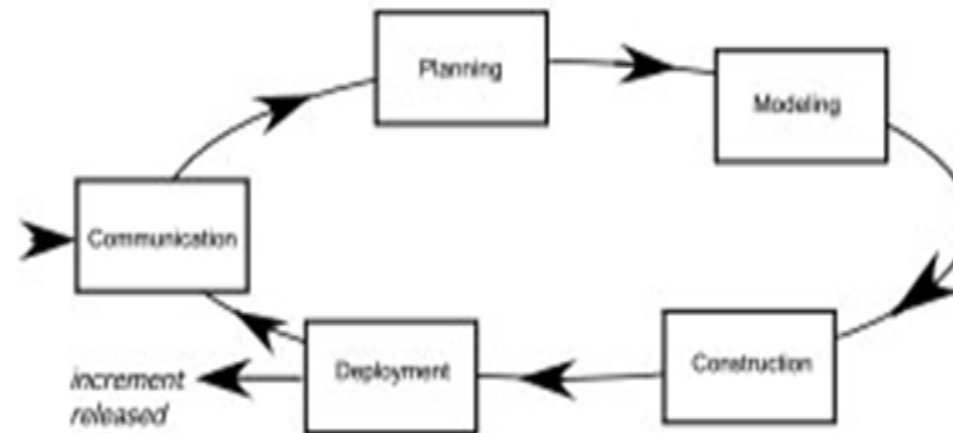
Iterative process flow

- An iterative process flow repeats one or more of the activities before proceeding to the next.



An evolutionary process flow

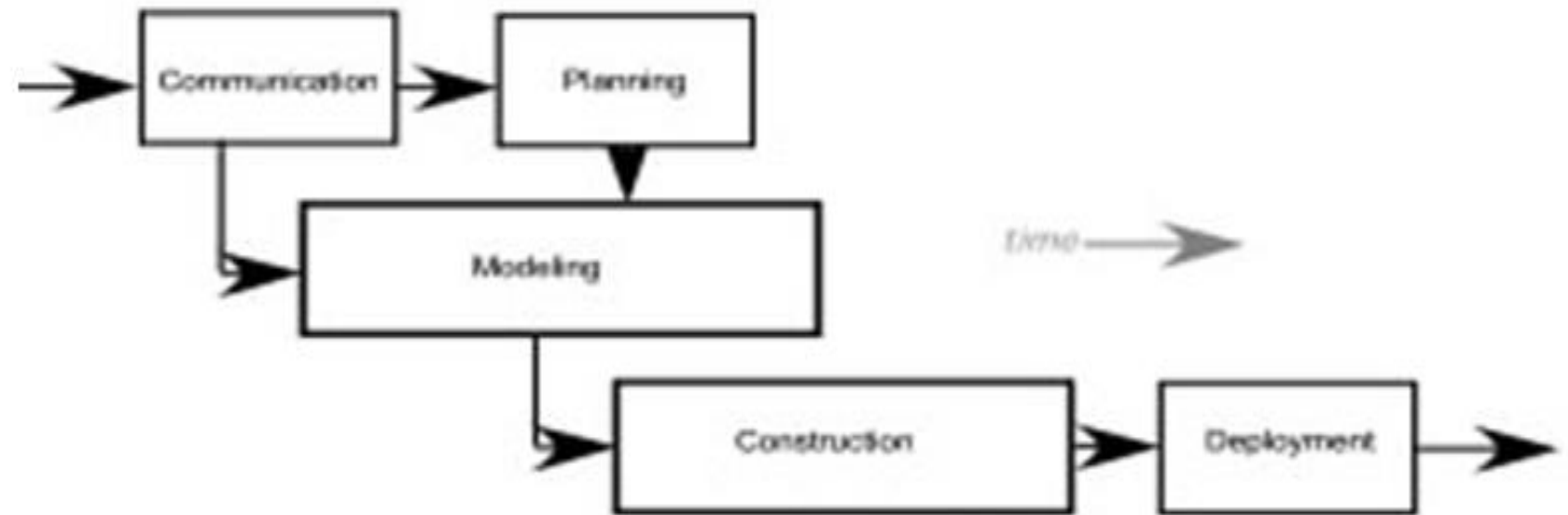
- An evolutionary process flow executes the activities in a circular manner. Each circuit leads to a more complete version of the software.



(c) evolutionary process flow

A parallel process flow

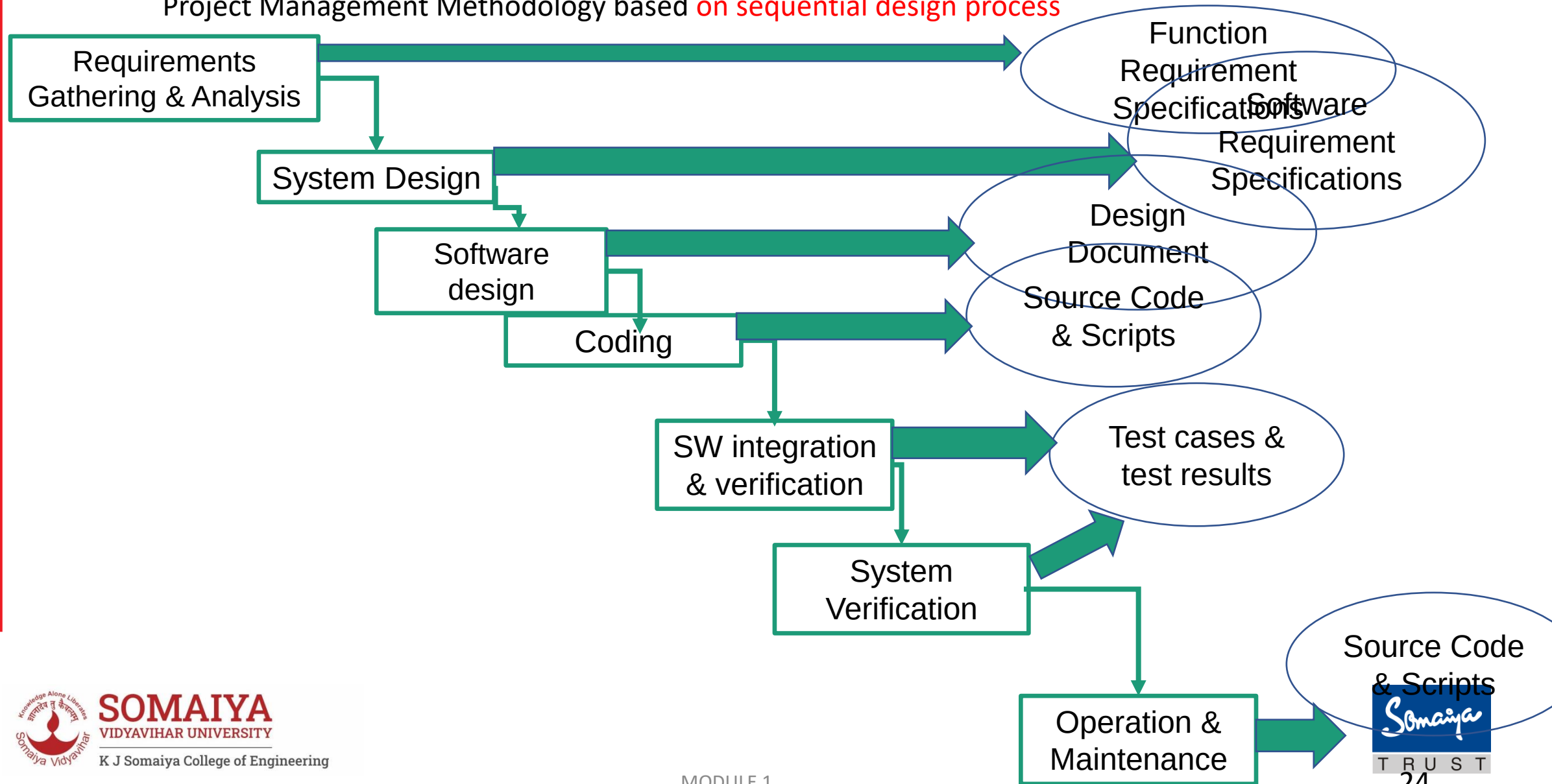
- A parallel process flow executes one or more activities in parallel with other activities (modeling for one aspect of the software in parallel with construction of another aspect of the software).



(d) parallel process flow

Waterfall Model

Project Management Methodology based on sequential design process



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Waterfall Model

Documents are produced at each stage phase

Identical and compliance to other engineering process models

All requirements must be gathered & analyzed at the start (not practical)

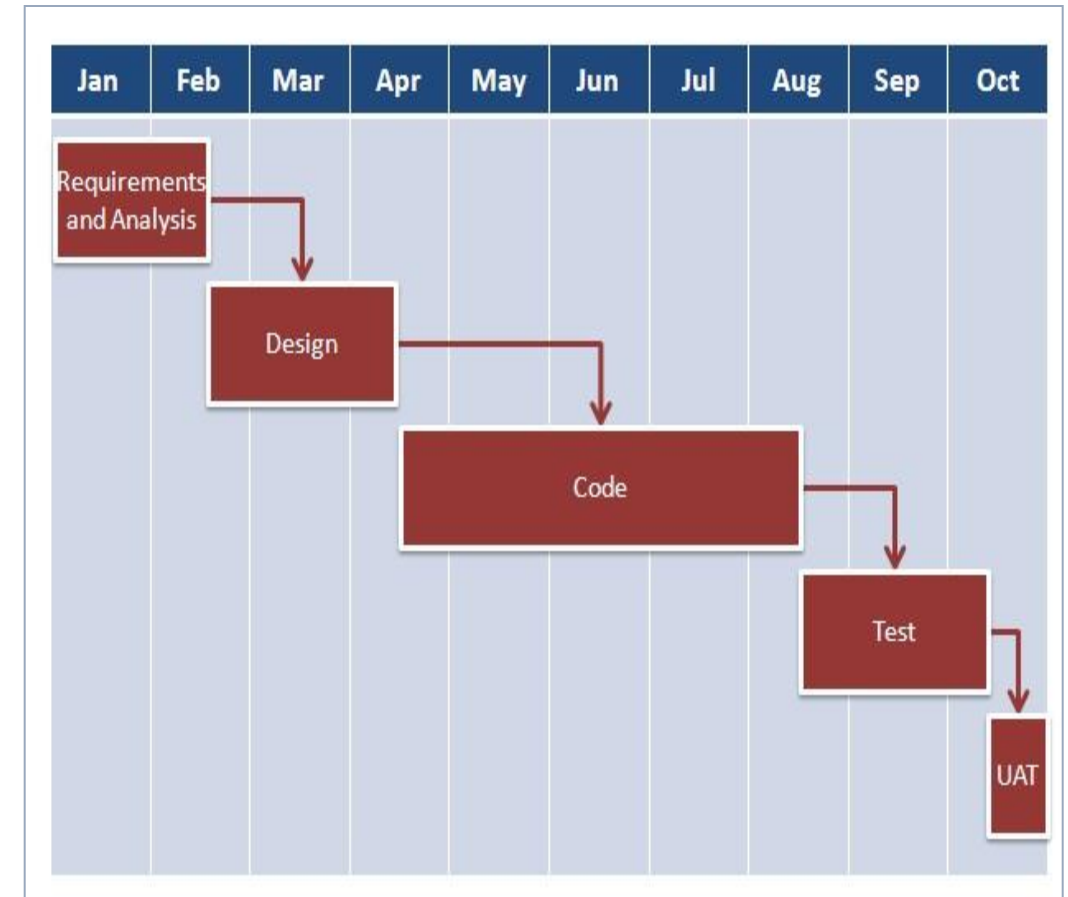
Phase-wise focus

Process restrictions encourages problem postponement end up in bad system

Design becomes inflexible

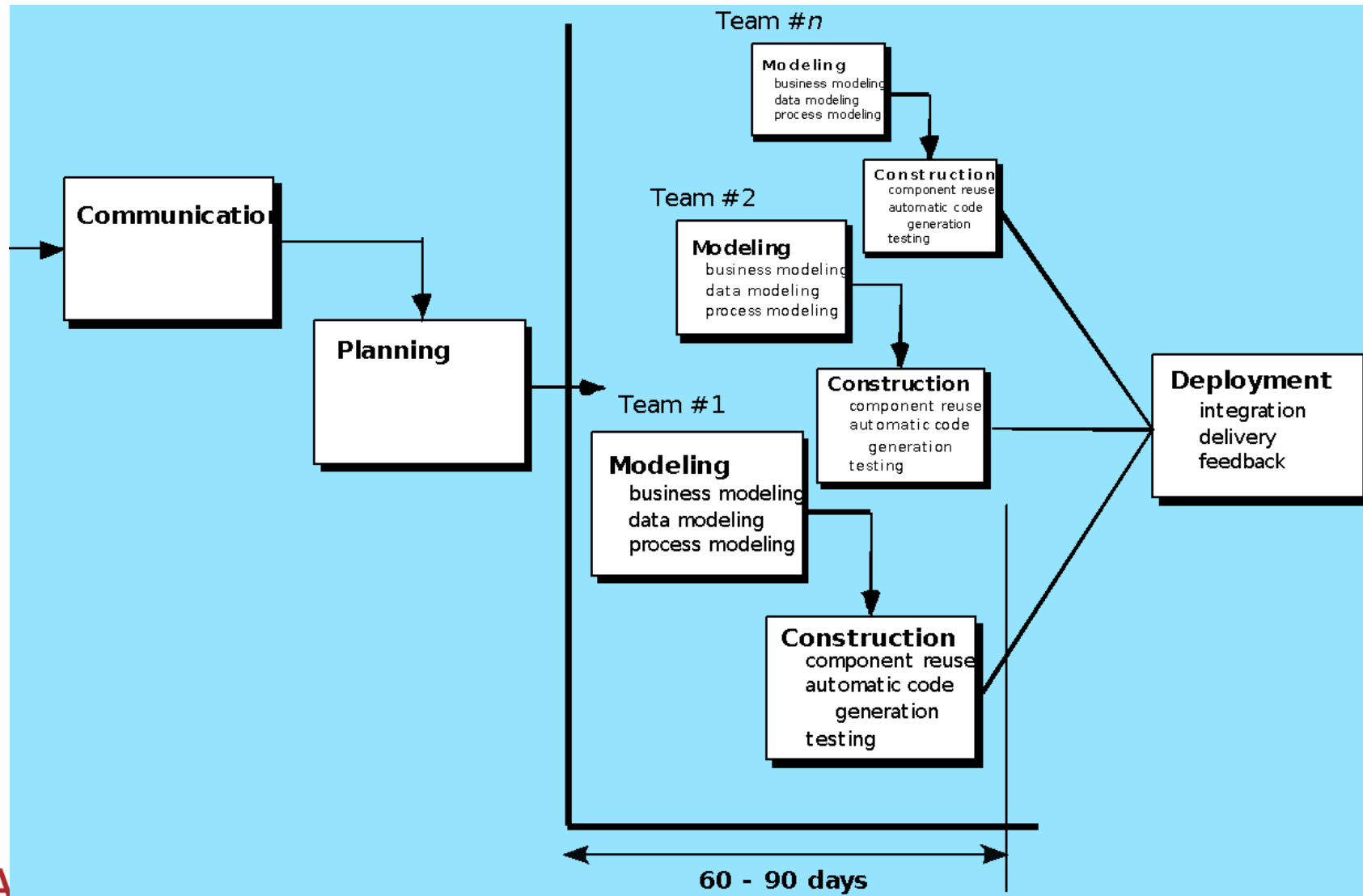
When to use the waterfall model?

- The project requirements are clear, unambiguous, and detailed
- The product definition is stable
- The tech stack is predefined and not dynamic
- Extensive resources with the necessary expertise are available
- The project is short



Project Plan

The RAD Model



SOMA

VIDYAVILLAR UNIVERSITY

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Rapid Application Development (RAD) Model

Delivery is available in short duration (within @ 60- 90 days)

Multiple teams work in parallel in activities such as modelling , construction of different modules (sub systems)

May be inconsistent

Integration could be difficult

When to use RAD Model?

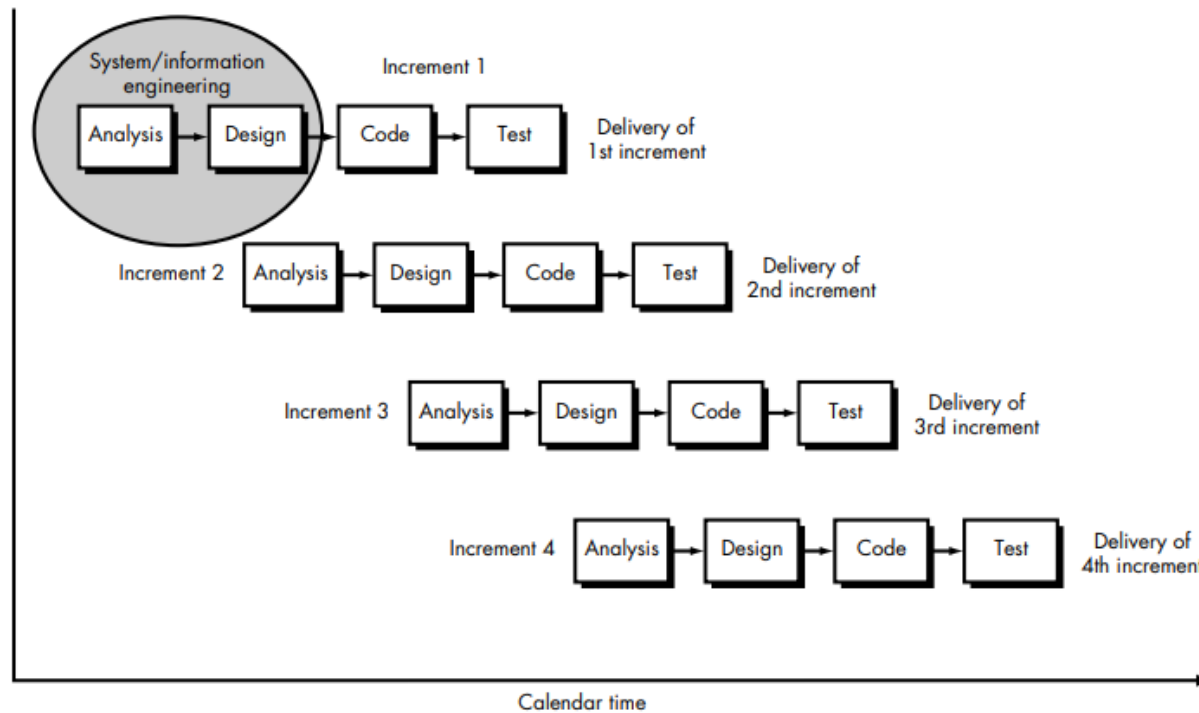
- When the customer has well-known requirements
- The user is involved throughout the life cycle
- The project can be time-boxed, the functionality delivered in increments, high performance is not required, low technical risks are involved and the system can be modularized.

Limitations of RAD model

- For large but scalable projects, RAD requires sufficient human resources to create the right number of RAD teams.
- RAD requires developers and customers who are committed to the rapid-fire activities necessary to get a system complete in a much abbreviated time frame. If commitment is lacking from either constituency, RAD projects will fail.
- Not all types of applications are appropriate for RAD. If a system cannot be properly modularized, building the components necessary for RAD will be problematic. If high performance is an issue and performance is to be achieved through tuning the interfaces to system components, the RAD approach may not work.
- RAD is not appropriate when technical risks are high. This occurs when a new application makes heavy use of new technology or when the new software requires a high degree of interoperability with existing computer programs.

Incremental Process model

FIGURE 2.7
The
incremental
model



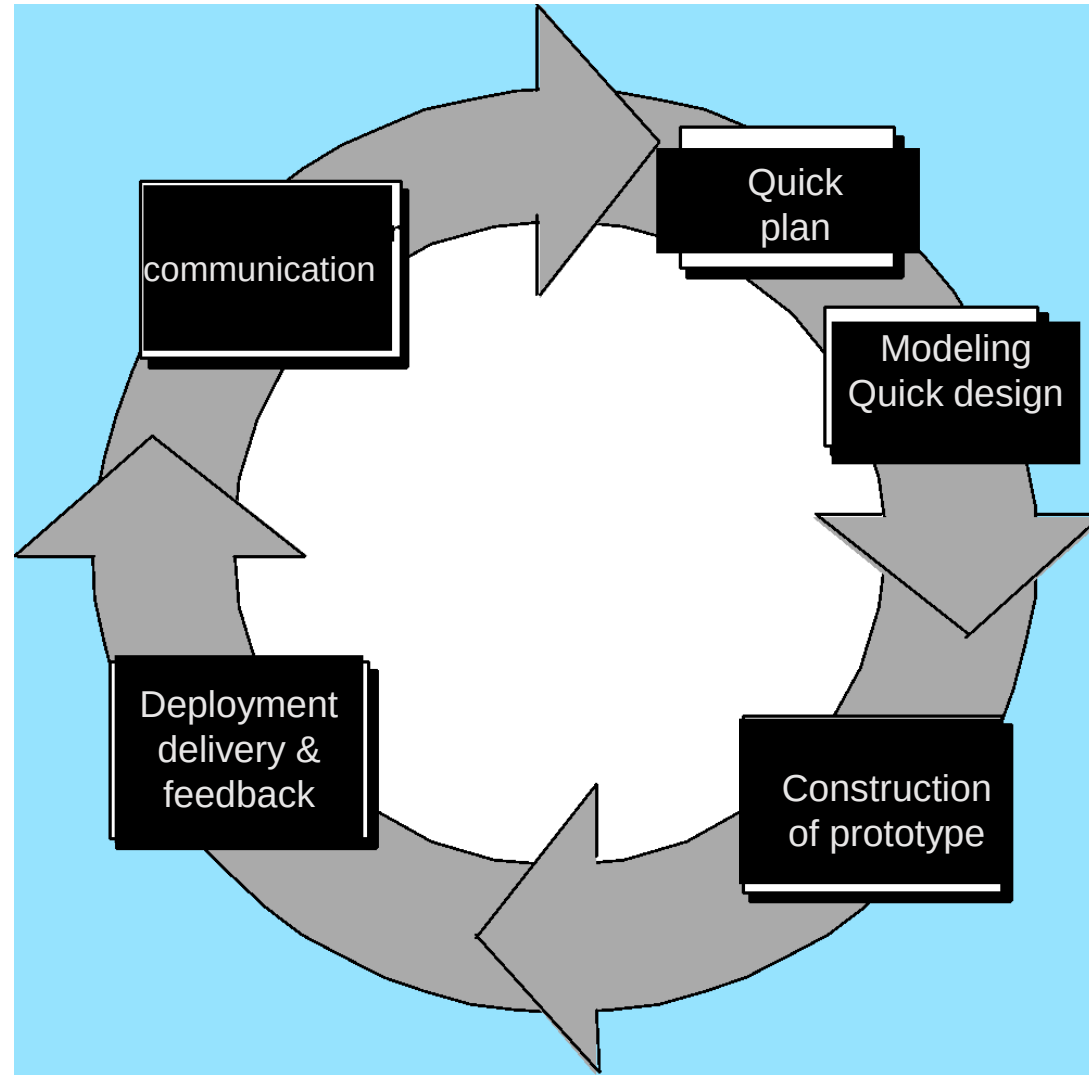
Incremental Process model

- The incremental model combines elements of linear and parallel process flow

Ex. **word-processing software**

- Basic file management, editing, and document production functions in the first increment;
- More sophisticated editing and document production capabilities in the second increment
- spelling and grammar checking in the third increment;
- Advanced page layout capability in the fourth increment

Evolutionary Models: Prototyping



These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

K J Somaiya College of Engineering

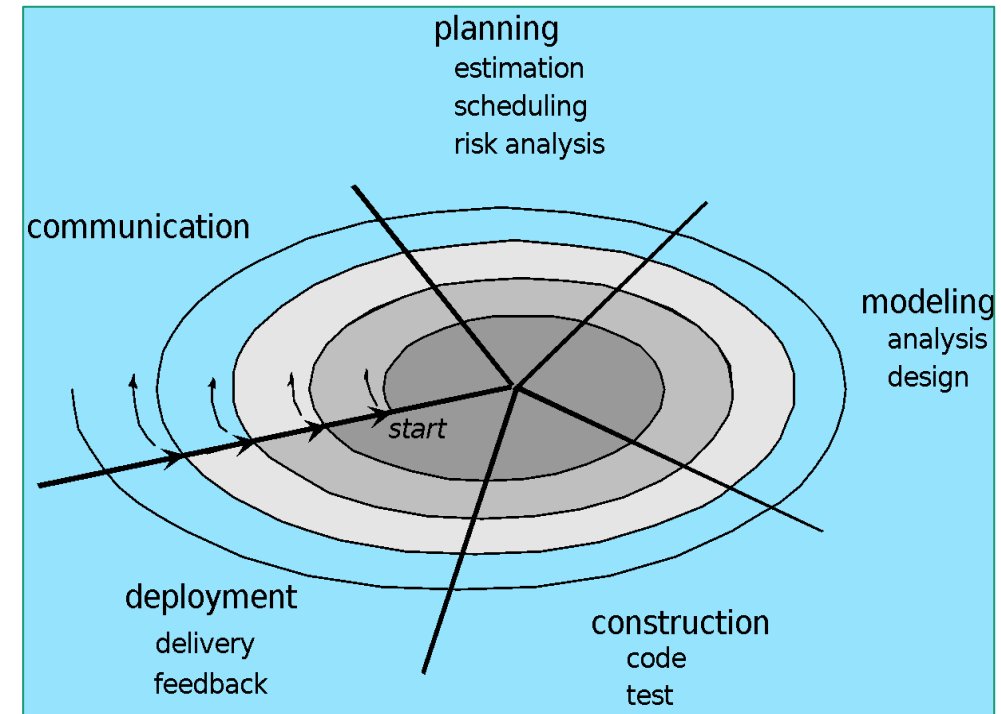
Evolutionary Prototype Model

Could be used for understanding
Requirements
Functioning
Testing new technology

The code/ product MAY NOT be used in the final product

Boehm's Spiral model

- **Spiral model** is one of the most important Software Development Life Cycle models, which provides support for Risk Handling.
- In its diagrammatic representation, it looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. Each loop of the spiral is called a Phase of the software development process
- The spiral model combines the idea of iterative development with the systematic, controlled aspects of the waterfall model. This Spiral model is a combination of iterative development process model and sequential linear development model i.e. the waterfall model with a very high emphasis on risk analysis



Spiral Model - Design

The spiral model has four phases. A software project repeatedly passes through these phases in iterations called Spirals.

Identification

This phase starts with gathering the business requirements in the baseline spiral. In the subsequent spirals as the product matures, identification of system requirements, subsystem requirements and unit requirements are all done in this phase.

This phase also includes understanding the system requirements by continuous communication between the customer and the system analyst. At the end of the spiral, the product is deployed in the identified market.

Design

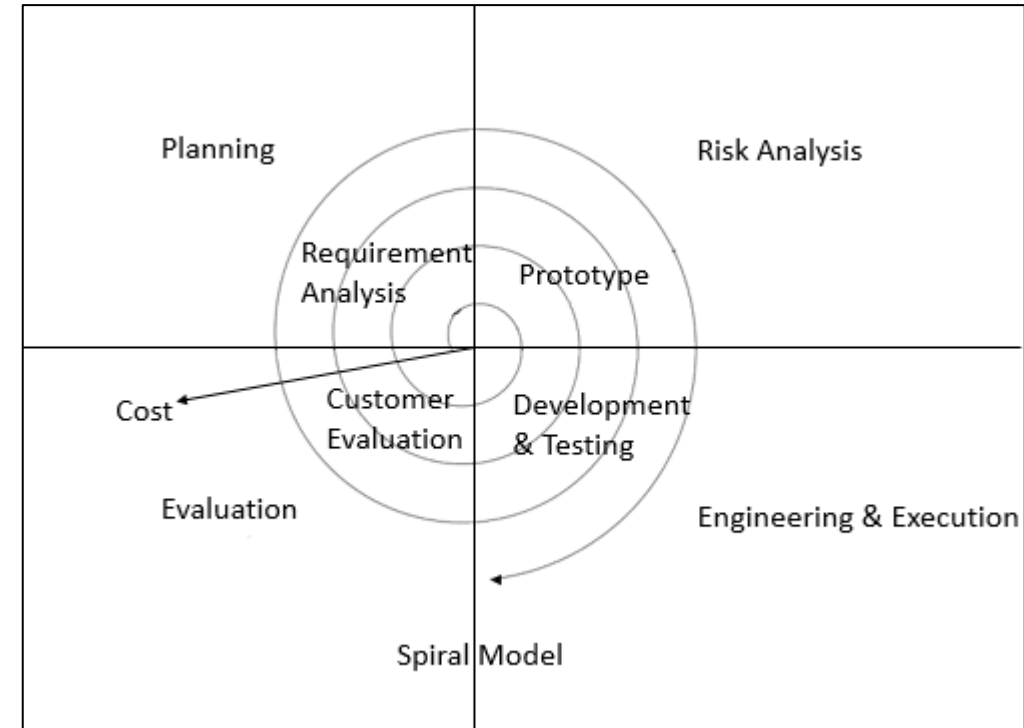
The Design phase starts with the conceptual design in the baseline spiral and involves architectural design, logical design of modules, physical product design and the final design in the subsequent spirals.

Construct or Build

The Construct phase refers to production of the actual software product at every spiral.

Evaluation and Risk Analysis

Risk Analysis includes identifying, estimating and monitoring the technical feasibility and management risks



Boehm's Spiral Model

Documents and product are available after each iteration

Improves & adds functionalities after each cycle

Requires ALL types of human resources at regular frequency

Spiral Model - Design

- Projects in which frequent releases are necessary
- Projects in which changes may be required at any time
- Medium to high risk projects
- Projects in which cost and risk analysis is important

Agile

- Effective (rapid and adaptive) **response to change** (team members, new technology, requirements)
- Effective **communication** in structure and attitudes among all team members, technological and business people, software engineers and managers
- Involving **the customer into the team**
- Working as a **team**
- Planning in an uncertain world has its limits and plan must be **flexible**
- Organizing a **team** so that it **is in control of the work performed**
- Eliminate all but the **most essential work products** and keep them **lean**
- Emphasize an **incremental delivery** strategy as opposed to intermediate products that gets working software to the customer as rapidly as feasible.
- **Rapid**, incremental delivery of software
- The development guidelines stress **delivery over analysis and design** although these activities are not discouraged, and **active and continuous communication** between developers and customers



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Benefits of Agility

Relatively easy to accommodate a change when a team is gathering requirements early in a project

The costs of doing this work are minimal

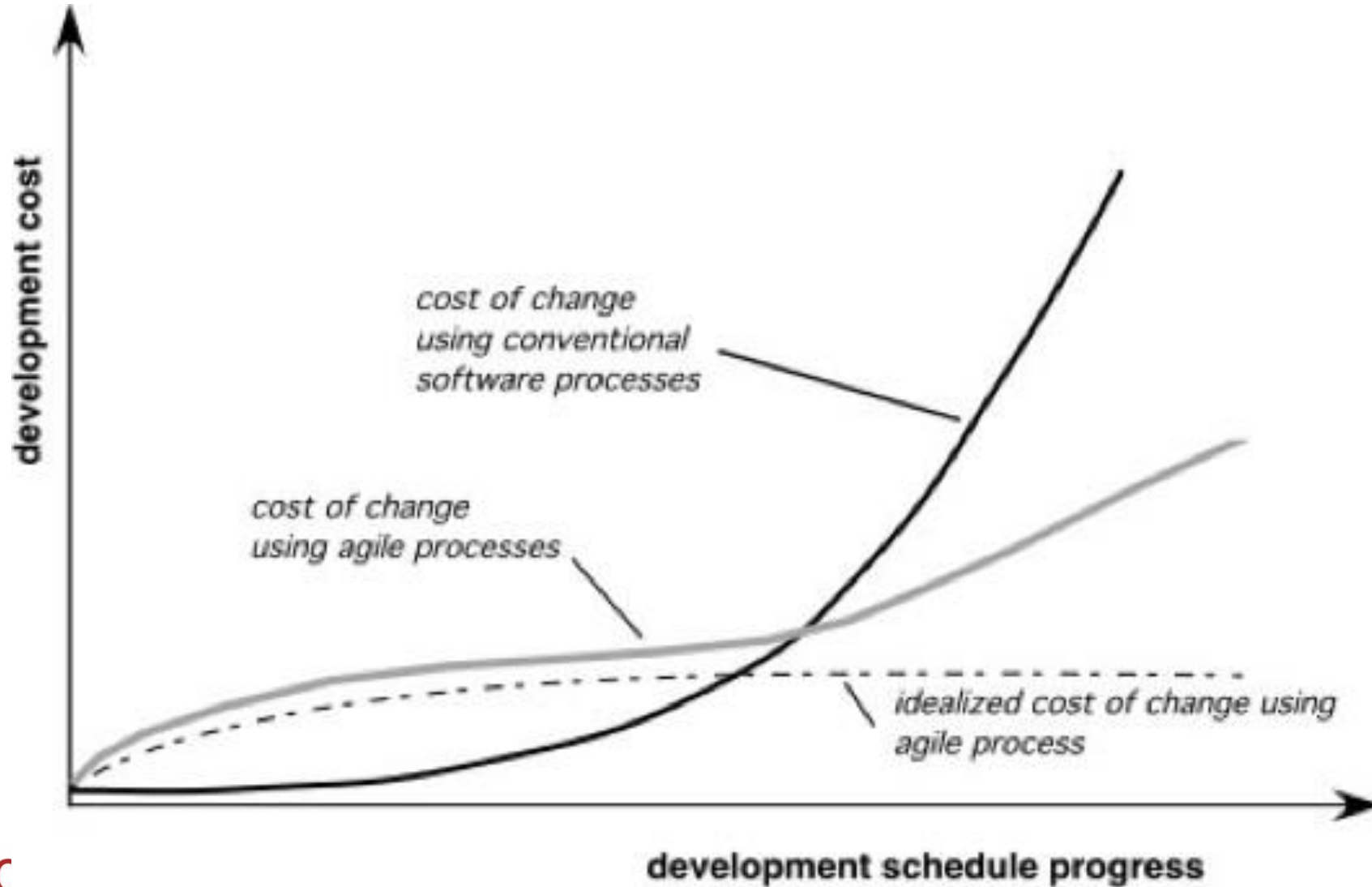
A well-designed agile process may “flatten” the cost of change curve by coupling incremental delivery with agile practices such as continuous unit testing and pair programming.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Cost of Change in Agile Approach



Agile Process

- A **customer descriptions** of what is required (scenarios). Some assumptions:
- Recognizes that plans are short-lived
 - some requirements will persist
 - some will change
 - Customer priorities will change
- Develops software iteratively with a heavy **emphasis on construction** activities
 - Design and construction are interleaved, hard to say how much design is necessary before construction
 - Design models are proven as they are created
 - Analysis, design, construction and testing are not predictable
- Adapt as changes occur due to unpredictability
- Delivers multiple 'software increments', deliver an operational prototype or portion of an OS to collect customer feedback for adaption.

Principles of Agility

1. Highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Allow changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale
4. Business people and developers must work together daily throughout the project
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation

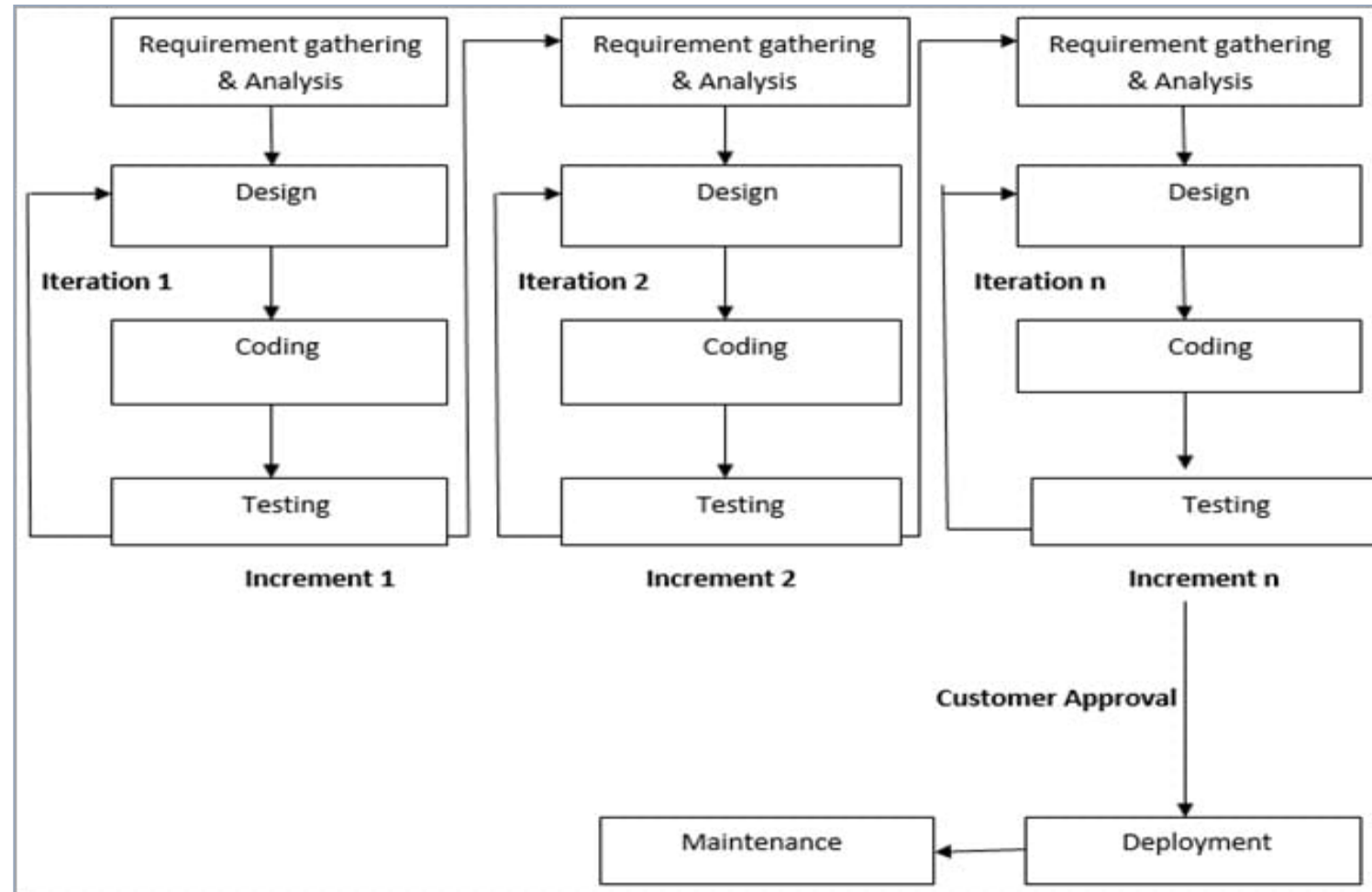
Principles of Agility

7. Working software is the primary measure of progress
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely
9. Continuous attention to technical excellence and good design enhances agility
10. Simplicity – the art of maximizing the amount of work not done – is essential
11. The best architectures, requirements, and designs emerge from self-organizing teams
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly

Agile model

- Agile SDLC model is a combination of iterative and incremental process models
- Agile Methods break the product into small incremental builds. These builds are provided in iterations.
- Each iteration typically lasts from about one to three weeks.

Agile Model



Advantages of Agile Model:

- It allows more flexibility to adapt to the changes.
- The new feature can be added easily.
- Customer satisfaction as the feedback and suggestions are taken at every stage.

Disadvantages of Agile Model:

- Agile needs experienced and highly skilled resources.
- If a customer is not clear about how exactly they want the product to be, then the project would fail.

Human factors in Agile Development Process

Development would be people focused than process

- **Competence** (talent, skills, knowledge)
- **Common focus** (focus on one goal , deliver a working software increment within time promised)
- **Collaboration** (peers and stakeholders)
- **Decision-making ability** (freedom to control its own destiny)
- **Fuzzy problem-solving ability** (ambiguity and constant changes, today problem may not be tomorrow's problem)
- **Mutual trust** and respect
- **Self-organization** (themselves for the work done, process for its local environment, the work schedule)

Agile Approach Extreme Programming

- The most widely used agile process
 - **XP Planning**
 - **XP Design**
 - **XP Coding**
 - **XP Testing**



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Extreme Programming (XP)

- The most widely used agile process

- **XP Planning**

- Begins with the listening, leads to creation of “user stories or use cases” that describes required output, features, and functionality. Customer assigns a value(i.e., a priority) to each story
- Agile team assesses each story and assigns a cost (development weeks. If more than 3 weeks, customer asked to split into smaller stories)
- Working together, stories are grouped for a deliverable increment next release.
- A commitment (stories to be included, delivery date and other project matters) is made. Three ways:
 1. Either all stories will be implemented in a few weeks,
 2. High priority stories first, or
 3. the riskiest stories will be implemented first.
- After the first increment “**project velocity**” (number of stories implemented during the first release) is used to help define subsequent delivery dates for other increments. Customers can add stories, delete existing stories, change values of an existing story, split stories as development work proceeds

Extreme Programming (XP)

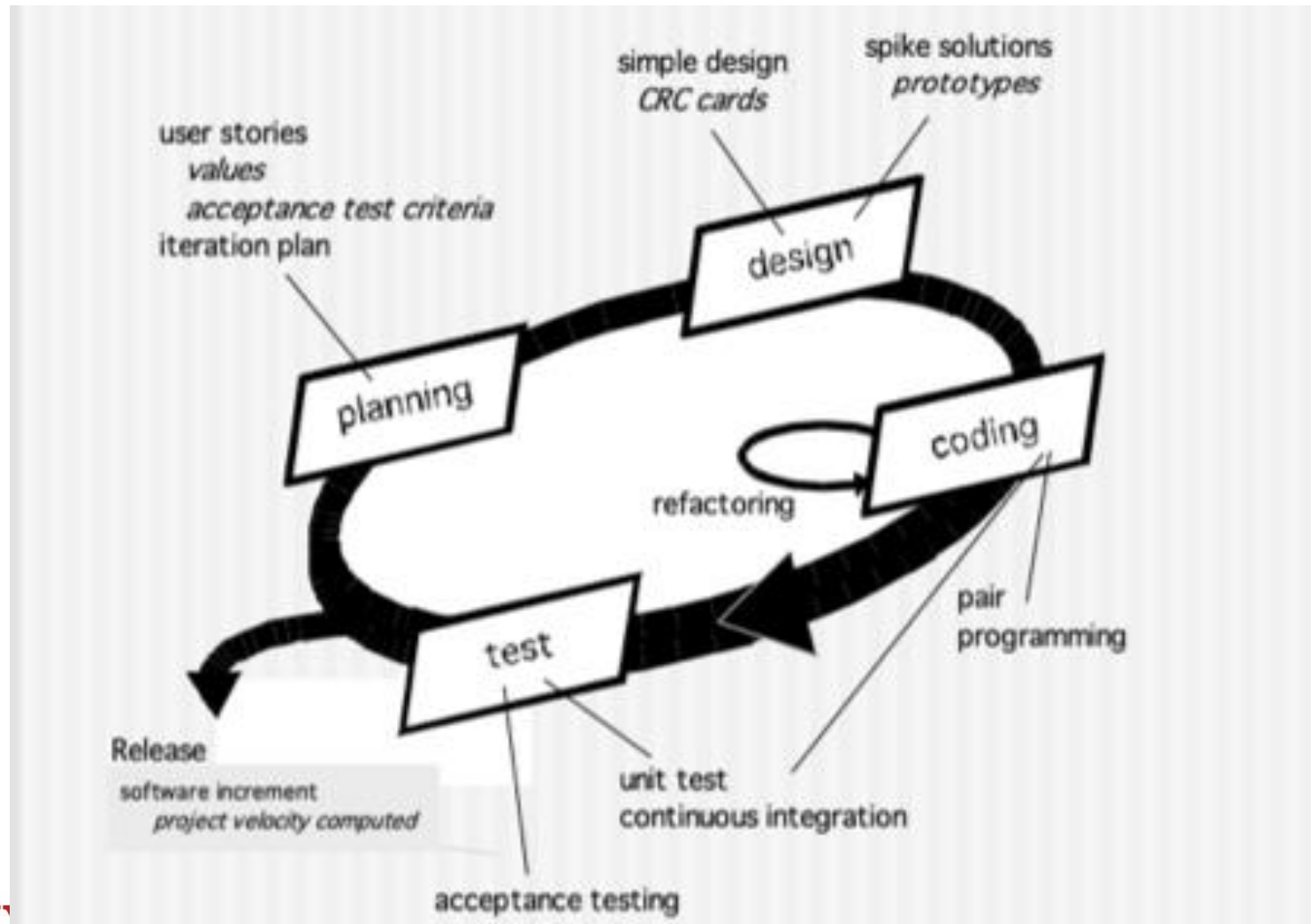
- **XP Design** (occurs both before and after coding as refactoring is encouraged)!
 - Follows the Keep It Simple(KIS) principle)Nothing more nothing less than the story.
 - Encourage the use of **CRC (class-responsibility-collaborator)** cards in an object-oriented context. The only design work product of XP. They identify and organize the classes that are relevant to the current software increment
 - For difficult design problems, suggests the creation of “spike solutions”—a design prototype for that portion is implemented and evaluated
 - Encourages “**refactoring**”—an iterative refinement of the internal program design. Does not alter the external behaviour yet improve the internal structure. Minimize chances of bugs. More efficient, easy to read

- **XP Coding**
- Recommends the construction of a unit test for a story before coding commences. So implementer can focus on what must be implemented to pass the test
- Encourages “pair programming”. Two people work together at one workstation. Real time problem solving, real time review for quality assurance. Take slightly different roles
- **XP Testing**
- All unit tests are executed daily and ideally should be automated. Regression tests are conducted to test current and previous components
- “Acceptance tests” are defined by the customer and executed to assess customer visible functionality

Merits & Limitations of XP

- **Requirements volatility:** customer is an active member of XP team, changes to requirements are requested informally and frequently
- **Conflicting customer needs:** different customers' needs to be assimilated. Different vision or beyond their authority
- **Requirements are expressed informally:** Use stories and acceptance tests are the only explicit manifestation of requirements. Formal models may avoid inconsistencies and errors before the system is built. Proponents said changing nature makes such models obsolete as soon as they are developed
- **Lack of formal design:** XP deemphasizes the need for architectural design. Complex systems need overall structure to exhibit quality and maintainability. Proponents said incremental nature limits complexity as simplicity is a core value

Extreme Programming (XP)



Adaptive Software Development (ASD)

Focusing on human collaboration and team self-organization as a technique to build complex software and system

Distinguishing features

- **Mission-driven** planning
- **Component-based** focus
- Uses “**time-boxing**” focus of Project Management Concepts
- Explicit **consideration of risks**
- Emphasizes **collaboration** for requirements gathering
- Emphasizes “**learning**” throughout the process

Three Phases of ASD

- 1. Speculation:** project is initiated and adaptive cycle planning is conducted
 - Adaptive cycle planning uses project initiation information- the customer's mission statement, project constraints (e.g. delivery date), and basic requirements to define the set of release cycles (increments) that will be required for the project.
 - Based on the information obtained at the completion of the first cycle, the plan is reviewed and adjusted so that planned work better fits the reality.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Three Phases of ASD

2. Collaborations are used to multiply their talent and creative output beyond absolute number.

It encompasses communication and teamwork, but it also emphasizes individualism, because individual creativity plays an important role in collaborative thinking

It is a matter of trust.

- assist without resentments
- work as hard as or harder than they do
- have the skill set to contribute to the work at hand
- communicate problems or concerns in a way that leads to effective action.

Three Phases of ASD

3. Learning: As members of ASD team begin to develop the components, the emphasis is on “learning”.

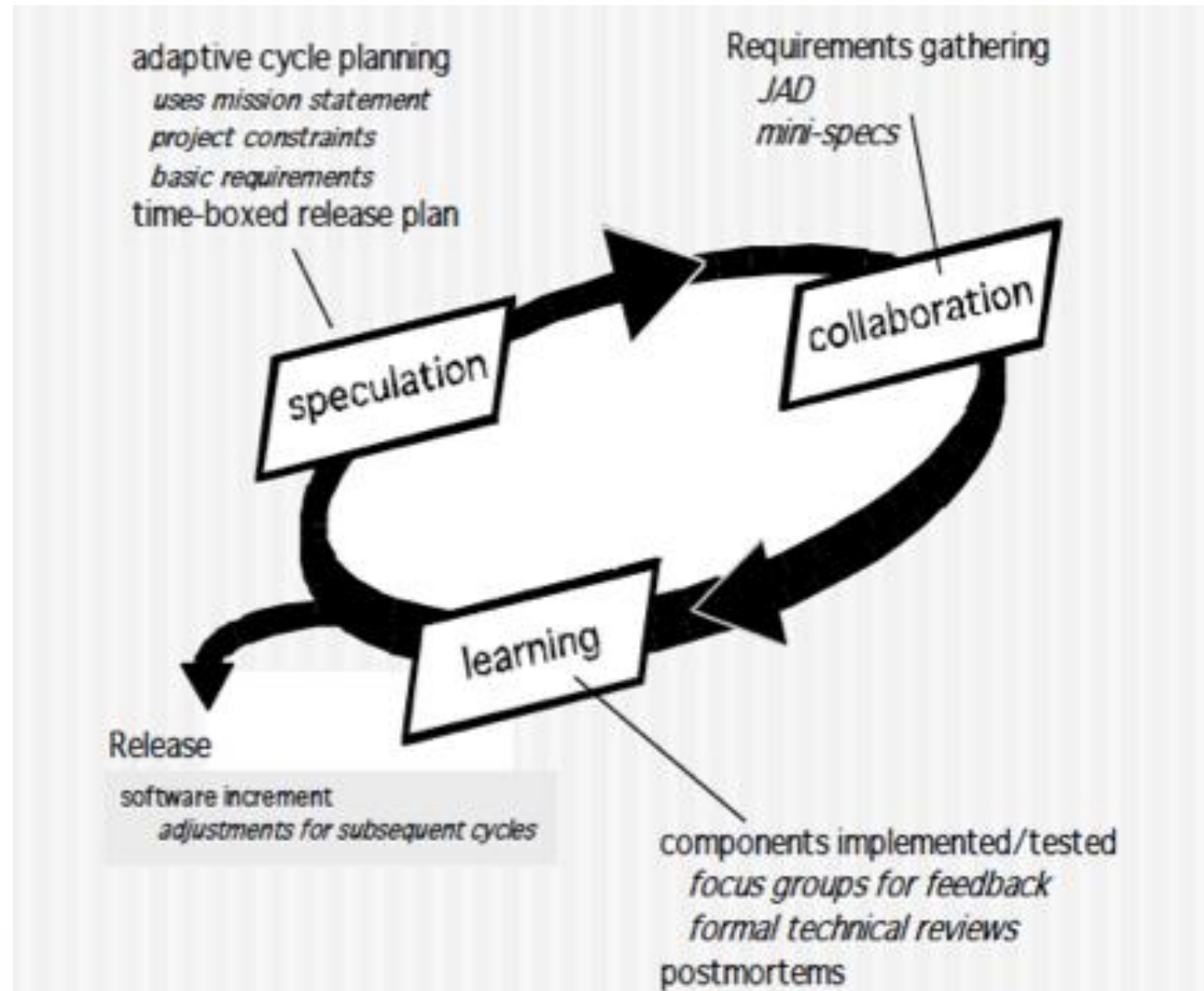
- software developers often overestimate their own understanding of the technology, the process, and the project and that learning will help them to improve their level of real understanding.
- Three ways: focus groups, technical reviews etc.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Three Phases of ASD



Dynamic Systems Development Method (DSDM)

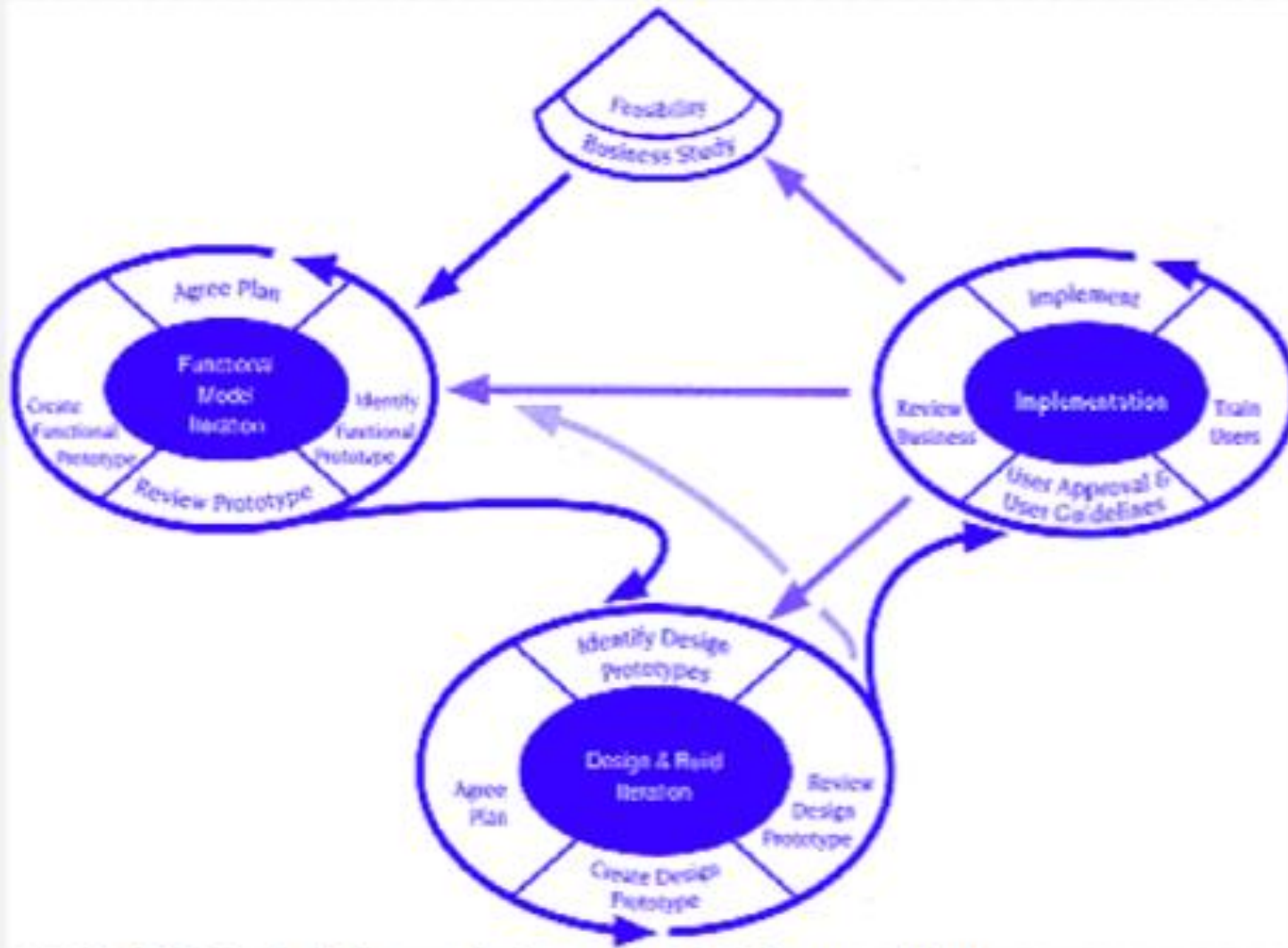
Provides a framework for building and maintaining systems which meet tight time constraints through the use of incremental prototyping in a controlled project environment

- DSDM—distinguishing features
- Similar in most respects to XP and/or ASD

- **Guiding principles**

1. Active user involvement is imperative.
2. DSDM teams must be empowered to make decisions
3. The focus is on frequent delivery of products
4. Fitness for business purpose is the essential criterion for acceptance of deliverables
5. Iterative and incremental development is necessary to converge on an accurate business solution
6. All changes during development are reversible
7. Requirements are baselined at a high level
8. Testing is integrated throughout the life-cycle

DSDM Life Cycle



DSDM Life Cycle (with permission of the DSDM consortium)

Scrum

Features

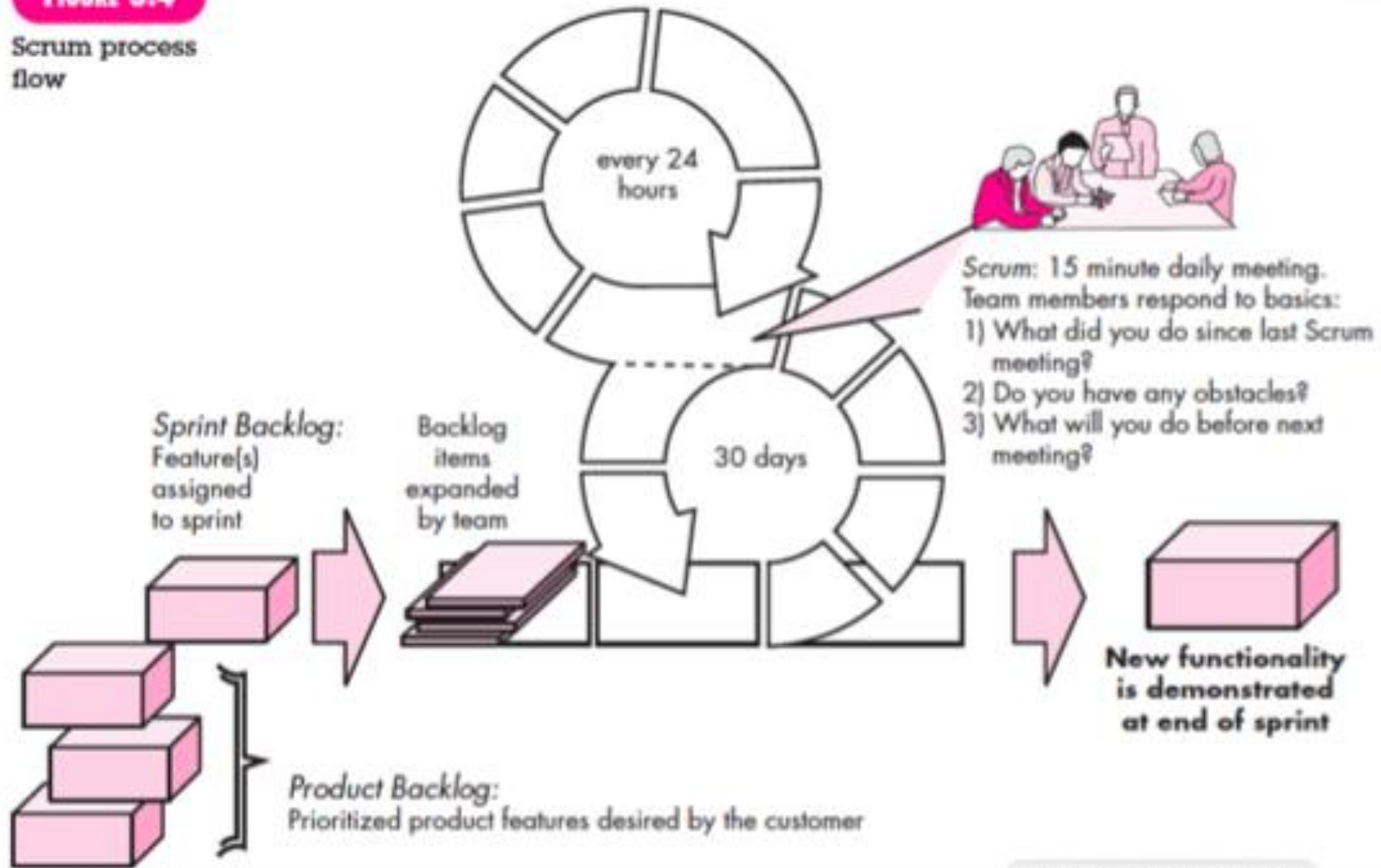
- Development work is partitioned into “**packets**”
- **Testing and documentation are on-going** as the product is constructed
- Work units occurs in “**sprints**” and is derived from a “**backlog**” of existing changing prioritized requirements
- **Sprints**: consist of work units that are required to achieve a requirement de-fined in the backlog that must be fit into a predefined time-box¹⁴ (typically 30 days). Changes (e.g., backlog work items) are not introduced during the sprint. Hence, the sprint allows team members to work in a short-term, but stable environment.
- **Backlog**—a prioritized list of project requirements or features that provide business value for the customer. Items can be added to the backlog at any time (this is how changes are introduced). The product manager assesses the backlog and updates priorities as required.

- Changes are not introduced in sprints (short term but stable) but in backlog
- **Meetings are very short** (15 minutes daily) (what did you do since last meeting? What obstacles are you encountering? What do you plan to accomplish by next meeting?)
- **“demos”** are delivered to the customer with the time-box allocated. May not contain all functionalities. So customers can evaluate and give feedbacks.

Scrum

FIGURE 3.4

Scrum process flow



Adobe Acrobat Reader DC

Feature Driven Development (FDD)

Features:

Emphasis is on defining “**features**” which can be organized hierarchically
a **feature** “is a client-valued function that can be implemented in two weeks or less.”

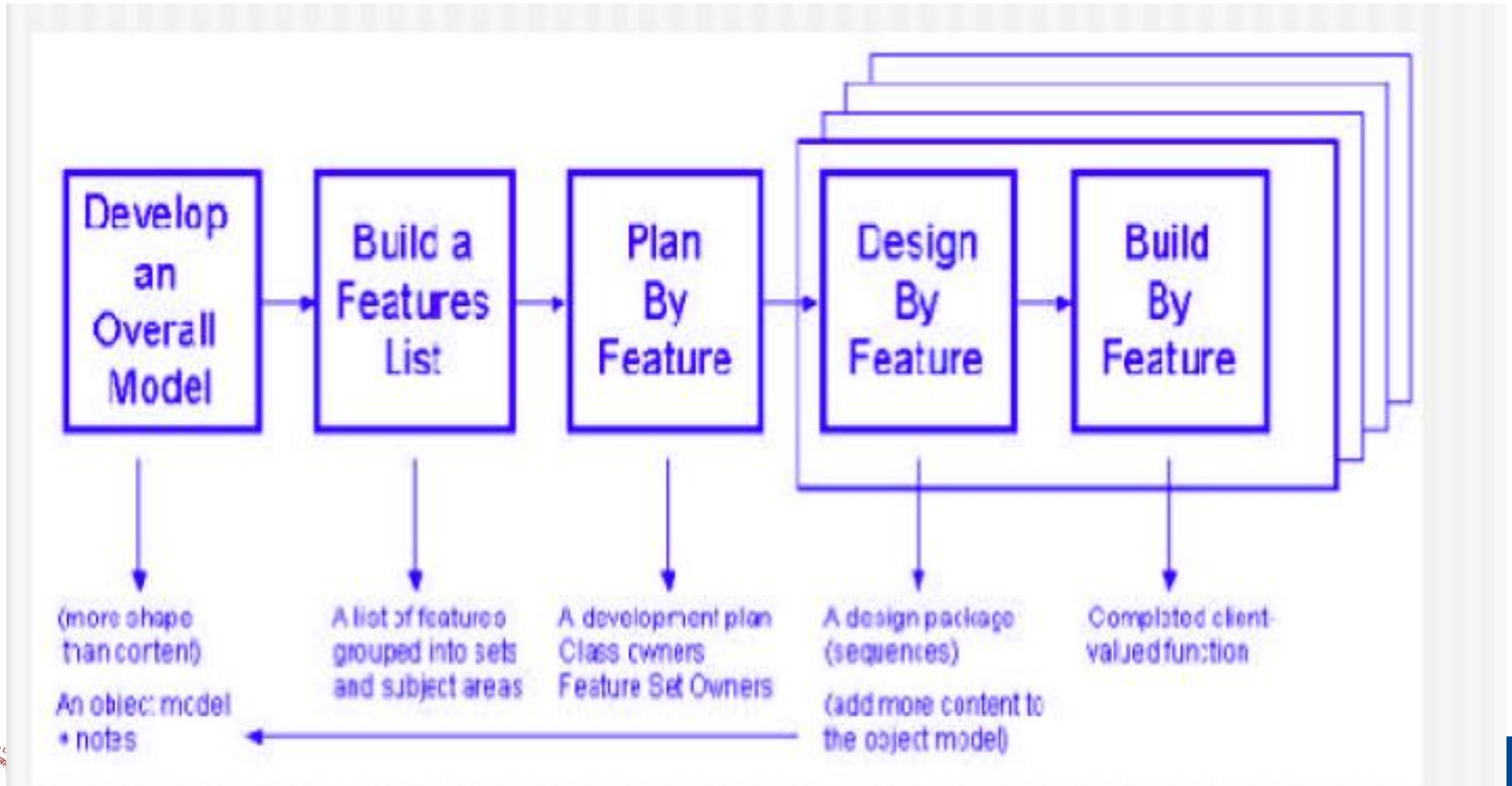
Uses a **feature template**

- <action> the <result> <by | for | of | to> a(n) <object>!
Object-a person, place, things(including roles, moments in time)
- E.g. Add the product to shopping cart.
- Display the technical-specifications of the product.
- Store the shipping-information for the customer.

A **features list** is created and “**plan by feature**” is conducted

Design and construction merge in FDD

FDD



Steps involved in Software Development

1. Identification of Problem
2. Problem Definition
3. Exploring approach(s) to solve the problem
4. Penning down/ finalizing the approach; listing the steps
5. Development
6. Testing
7. Deployment
8. Maintenance

These steps are iterative and some of them run parallel

Software Engineering

Module 1.2



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



The Product and the Process:

- Understanding software process
- Process metric
- CMM Levels

Software Process

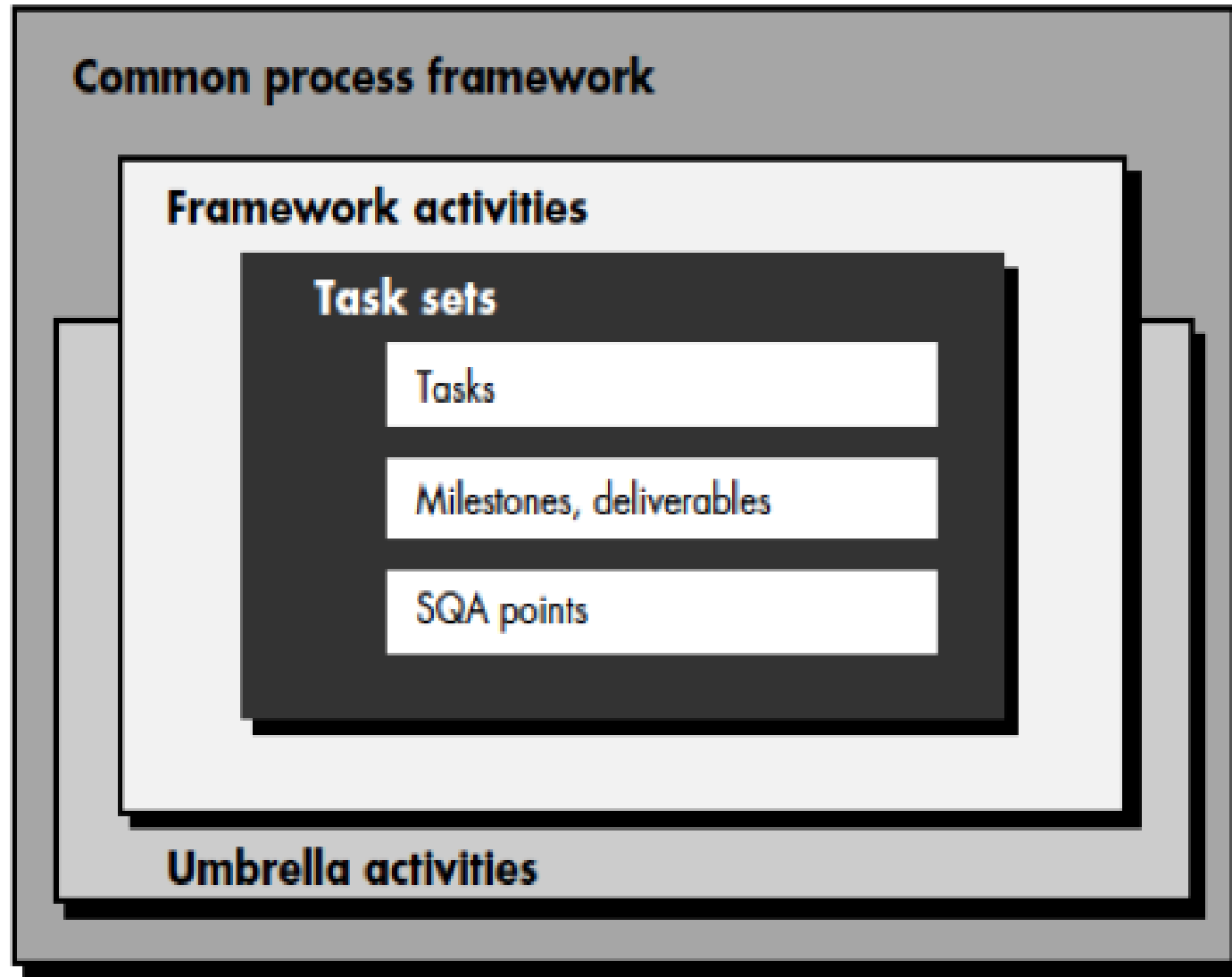
Characterized by:

- Common process framework established by defining a small number of framework activities that are applicable to all software projects, regardless of the their size or complexity

Consists:

- Framework Activities which include:
 - Task sets, represented by Tasks, Milestones & Deliverables , SQA points
 - Umbrella Activities

Process Framework activities



Process Framework activities

- Work tasks
- Work products
- Milestones & Deliverables
- Quality Assurance Check points

The following *generic process framework* is applicable to the vast majority of S/W projects.

- **Communication:** involves heavy communication with the customer (and other stakeholders) and encompasses requirements gathering.
- **Planning:** Describes the technical tasks to be conducted, the risks that are likely, resources that will be required, the work products to be produced and a work schedule.
- **Modeling:** encompasses the creation of models that allow the developer and customer to better understand S/W req. and the design that will achieve those req.
- **Construction:** combines code generation and the testing required uncovering errors in the code.
- **Deployment:** deliver the product to the customer who evaluates the delivered product and provides feedback.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Process Umbrella activities

Typical Umbrella activities include:

- Software project tracking and control
- Formal technical reviews
- Software quality assurance
- Software configuration management
- Document preparation and production
- Reusability management
- Measurement
- Risk management

Umbrella activities are applied throughout the software process

Software Metrics

- A software metric is a measure of software characteristics which are measurable or countable. Software metrics are valuable for many reasons, including measuring software performance, planning work items, measuring productivity, and many other uses.
- Within the software development process, many metrics are that are all connected. Software metrics are similar to the four functions of management: Planning, Organization, Control, or Improvement.

Classification of Software Metrics

Software metrics can be classified into two types as follows:

- **Product Metrics:** These are the measures of various characteristics of the software product. The two important software characteristics are:
 - Size and complexity of software.
 - Quality and reliability of software.

These metrics can be computed for different stages of SDLC.

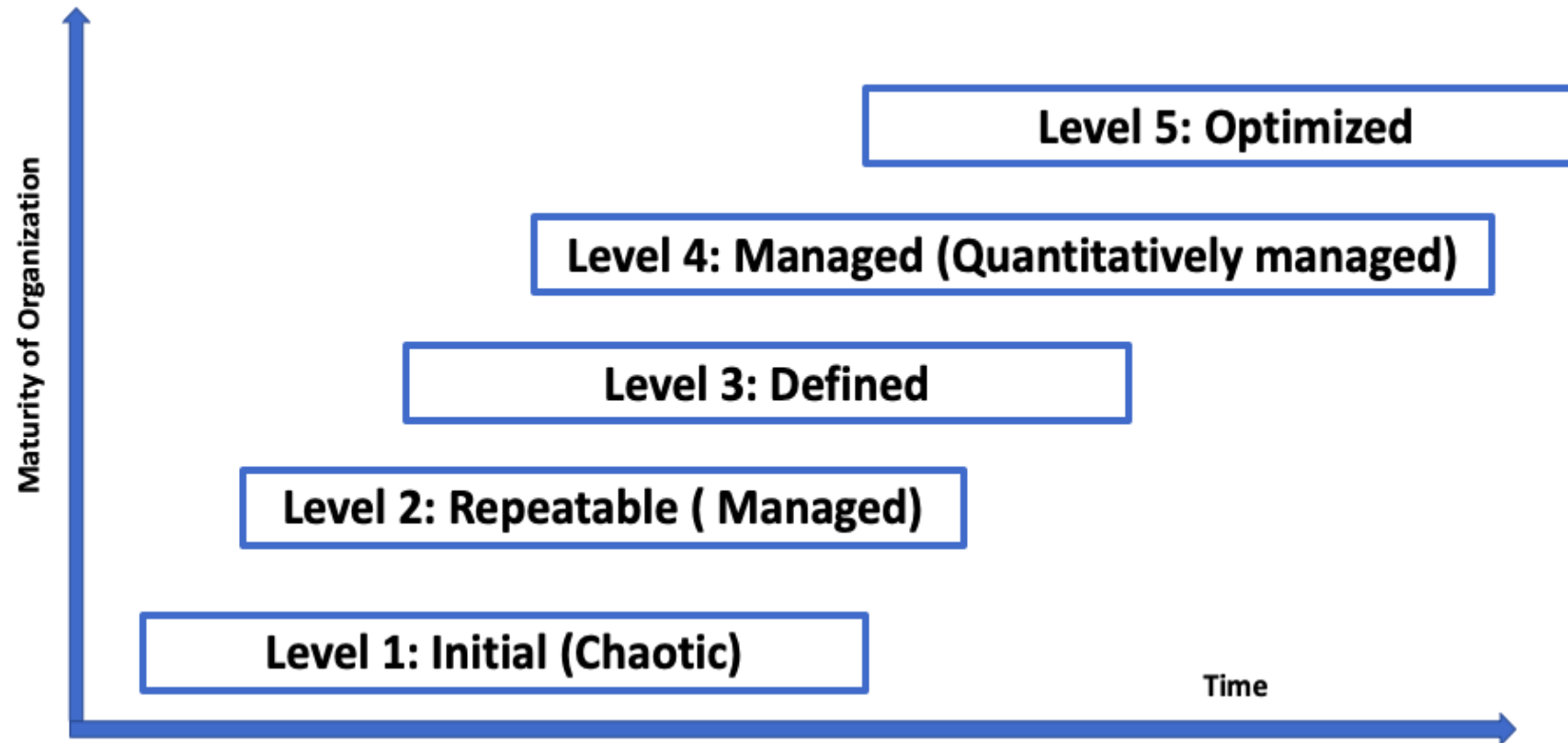
- **Process Metrics:** These are the measures of various characteristics of the software development process. For example, the efficiency of fault detection. They are used to measure the characteristics of methods, techniques, and tools that are used for developing software

Advantages of Software Metrics

- Reduction in cost or budget.
- It helps to identify the particular area for improvising.
- It helps to increase the product quality.
- Managing the workloads and teams.
- Reduction in overall time to produce the product,.
- It helps to determine the complexity of the code and to test the code with resources.
- It helps in providing effective planning, controlling and managing of the entire product.

Capability Maturity Model (CMM) Levels

The Capability Maturity Model (CMM) is a procedure used to develop and refine an organization's software development process.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Level 1 Initial

- At level 1, the process is usually chaotic and ad- hoc
- Few processes are defined, and success depends on individual effort.
- A capability is characterized on the basis of the individuals and not of the organization
- Progress not measured
- Products developed are often schedule and over budget

Level 2 Managed

- Basic project management processes are established to track cost, schedule, and functionality. The necessary process discipline is in place to repeat earlier successes on projects with similar applications.
 - Requirement Management
 - Estimate project parameters like cost, schedule, and functionality
 - Measure actual progress
 - Develop plans and process
 - Software project standards are defined
 - Identify and control products, problem reports changes, etc.
 - Processes may differ between projects

Level 3: Defined.

- The software process for both management and engineering activities is documented, standardized, and integrated into an organization-wide software process.
- All projects use a documented and approved version of the organization's process for developing and supporting software. This level includes all characteristics defined for level 2
 - Clarify customer requirements
 - Solve design requirements, develop an implementation process
 - Makes sure that product meets the requirements and intended use
 - Analyze decisions systematically
 - Rectify and control potential problems

- **Level 4: Managed (Quantitatively managed).** Detailed measures of the software process and product quality are collected. Both the software process and products are quantitatively understood and controlled using detailed measures. This level includes all characteristics defined for level 3.
- Manages the project's processes and sub-processes statistically
- Understand process performance, quantitatively manage the organization's project

- **Level 5: Optimizing.** Continuous process improvement is enabled by quantitative feedback from the process and from testing innovative ideas and technologies. This level includes all characteristics defined for level 4
- Detect and remove the cause of defects early
- Identify and deploy new tools and process improvements to meet needs and business objectives



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Key Process Areas (KPA)

•The SEI (Software Engineering Institute) has associated key process areas (KPAs) with each of the maturity levels which describe those software engineering functions (e.g., software project planning, requirements management) that must be present to satisfy good practice at a particular level.

- Characteristics Of KPAs**

- Goals**—the overall objectives that the KPA must achieve.

- Commitments**—requirements (imposed on the organization) that must be met to achieve the goals or provide proof of intent to comply with the goals.

- **Abilities**—those things that must be in place (organizationally and technically) to enable the organization to meet the commitments.

- Activities**—the specific tasks required to achieve the KPA function.

- Methods for monitoring implementation**—the manner in which the activities are monitored as they are put into place.

- Methods for verifying implementation**—the manner in which proper practice for the KPA can be verified.

1.3 Software Planning & Estimation(7/8/2023)

- Product Metrics estimations : LOC, FP, COCOMO



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Software Product metrics

- Software project measures are tactical; project metrics and the indicators derived from them are used by a project manager and a software team to adapt project workflow and technical activities.
- The first application of project metrics on most software projects occurs during estimation.
- Metrics collected from past projects are used as a basis from which effort and time estimates are made for current software work.
- As a project proceeds, measures of effort and calendar time expended are compared to original estimates (and the project schedule). The project manager uses these data to monitor and control progress.

Used to :

1. **minimize the development schedule** by making the adjustments necessary to avoid delays and mitigate potential problems and risks.
2. **assess product quality** on an ongoing basis and, when necessary, modify the technical approach to improve quality.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Software Measurement

- **Direct measures** of the software engineering process include cost and effort applied.
- **Direct measures** of the product include **lines of code (LOC) produced, execution speed, memory size, and defects** reported over some set period of time.
- **Indirect measures** of the **product** include **functionality, quality, complexity, efficiency, reliability, maintainability**



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Types of Software product Metrics

1. Size-Oriented Metrics

- metrics derived by normalizing quality and/or productivity measures by considering the **size of the software** that has been produced
- a set of simple size-oriented metrics can be developed for each project:
 - Errors per KLOC (thousand lines of code).
 - Defects per KLOC.
 - \$ per LOC.
 - Page of documentation per KLOC.

In addition, other interesting metrics can be computed:

- Errors per person-month.
- LOC per person-month.
- \$ per page of documentation.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Size-Oriented Metrics

- Size = Kilo Lines of Code (KLOC)
- Effort = Person / month
- Productivity = KLOC / person-month
- Quality = Number of faults / KLOC
- Cost = \$ / KLOC
- Documentation = Pages of documentation / KLOC

Project Name	LOC	Effort	Cost (\$)	Doc. (pages)	Errors	Defects
ABC	10, 000	20	170	400	100	12
PQR	20, 000	60	300	1000	129	32
XYZ	35, 000	65	522	1290	280	87

Product Metrics

2. Function-oriented Metrics

- software metrics use a measure of the functionality delivered by the application
- Function points derived from direct measures of the information domain.
 1. **Number of user inputs.** Each user input that provides distinct application oriented data to the software
 2. **Number of user outputs.** Each user output that provides application oriented information to the user is counted. In this context output refers to reports, screens, error messages, etc.
 3. **Number of user inquiries.** An inquiry is defined as an on-line input that results in the generation of some immediate software response in the form of an on-line output.
 4. **Number of internal logical files (ILFs).** Each logical master file (i.e., a logical grouping of data that may be one part of a large database or a separate file) is counted.
 5. **Number of external interfaces.** Each external interface file is a logical grouping of data that resides external to the application but provides information that may be of use to the application

Product Metrics

FP calculation

Measurement parameter	Count		Weighting factor			
			Simple	Average	Complex	
Number of user inputs		×	3	4	6	=
Number of user outputs		×	4	5	7	=
Number of user inquiries		×	3	4	6	=
Number of files		×	7	10	15	=
Number of external interfaces		×	5	7	10	=
Count total	→					



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

$$FP = \text{Count total} * (0.65 + 0.01 * \sum Fi)$$

Software Measurement

The F_i ($i = 1$ to 14) are "complexity adjustment values" based on responses to the following questions

1. Does the system require reliable backup and recovery?
2. Are data communications required?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require on-line data entry?
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?
8. Are the master files updated on-line?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

• Each of these questions is answered using a scale that ranges from

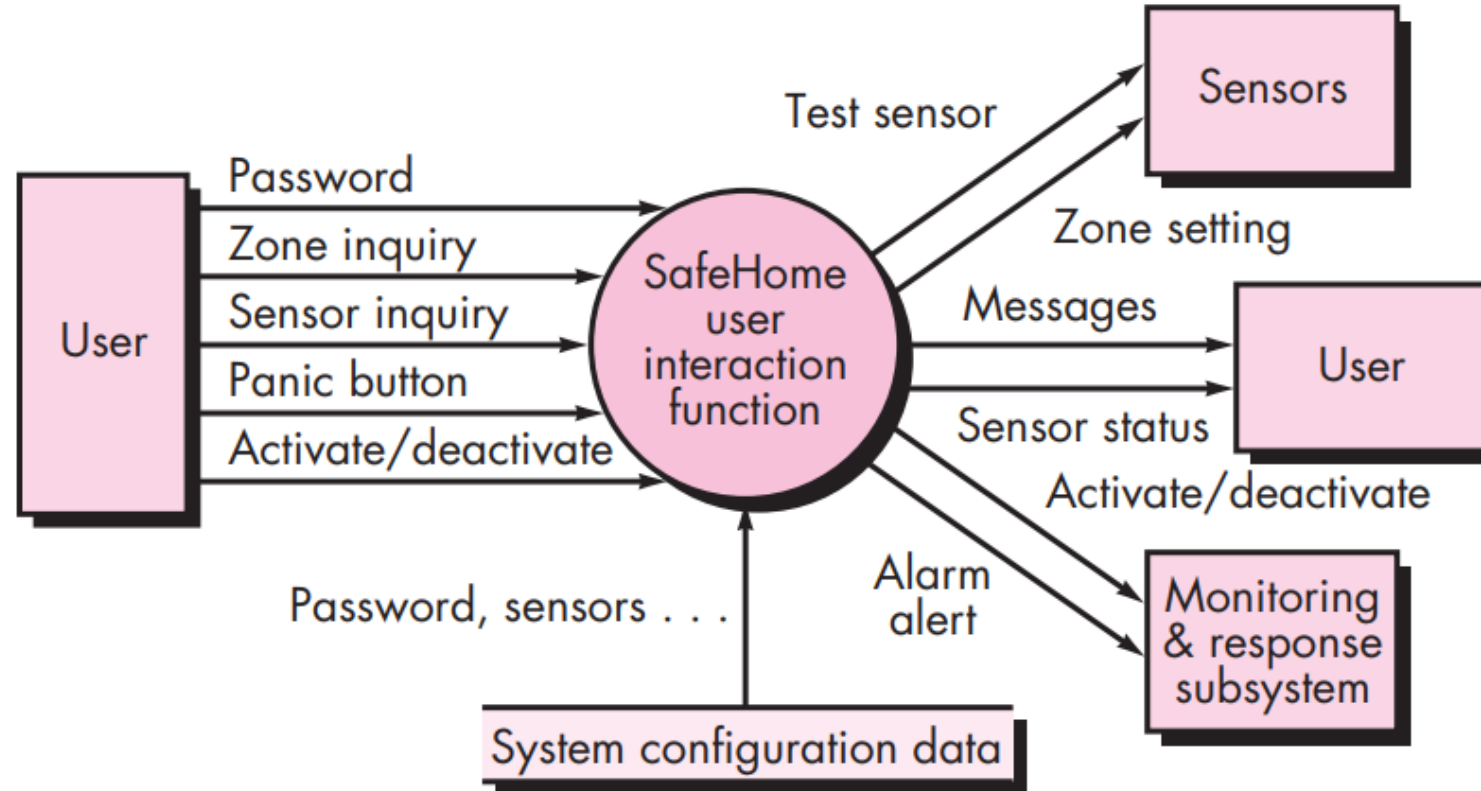
0 (not important or applicable) to 5 (absolutely essential)



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Example –FP calculation –Safe home software



- **Three external inputs**—password, panic button, and activate/deactivate
- **two external inquiries**—zone inquiry and sensor inquiry.
- One ILF (system configuration file)
- **Two external outputs** (messages and sensor status)
- **four EIFs** (test sensor, zone setting, activate/deactivate, and alarm alert)

Example –FP calculation –Safe home software

Information Domain Value	Count		Weighting factor				
			Simple	Average	Complex		
External Inputs (EIs)	3	×	3	4	6	=	9
External Outputs (EOs)	2	×	4	5	7	=	8
External Inquiries (EQs)	2	×	3	4	6	=	6
Internal Logical Files (ILFs)	1	×	7	10	15	=	7
External Interface Files (EIFs)	4	×	5	7	10	=	20
Count total							50

$$FP = \text{Count total} * (0.65 + 0.01 * \sum Fi)$$

$$FP = 50 \times [0.65 + (0.01 \times 46)] = 56$$

$$\sum(F_i) \text{ is } 46$$



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Software Measurement

Given the following values, **compute function point** when all complexity adjustment factor (CAF) are average and weighting factors are

- User Input = 40 Simple
- User Output = 35 Complex
- User Inquiries = 30 Simple
- User Files = 8 Average
- External Interface = 6 Average

$$\text{Count Total} = 40 * 3 + 35 * 7 + 30 * 3 + 8 * 10 + 6 * 7 = 577$$

$$\Sigma(F_i) = 14 * 3 = 42 \quad (\text{total 14 } F_i \text{ Each is average on scale of 1-5 hence 3})$$

$$\text{FP} = \text{count total} [0.65 + 0.01 \Sigma(F_i)]$$

$$\begin{aligned} \text{FP} &= 577 * (0.65 + 0.42) \\ &= 617.39 \end{aligned}$$

Software Measurement

- Once function points have been calculated, they are used in a manner analogous to LOC as a way to normalize measures for software productivity, quality, and other attributes:
 - Errors per FP.
 - Defects per FP.
 - \$ per FP.
 - Pages of documentation per FP.
 - FP per person-month.

Reference:

1. Rajesh Narang, Software Engineering Principles & Practices, McGraw Hill, (318-329)
2. Roger S. Pressman Software Engineering Practitioner's Approach, McGraw Hill, 5th edition (89-91)

Software Product Estimation

COCOMO model

- **Constructive Cost Estimation Model**
- Basic Effort Estimation Model uses size of Code to estimate **man month effort** and **project duration in month**
- **Effort:** Amount of labour that will be required to complete a task. It is measured in person-months units.
- **Schedule:** Simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put. It is measured in the units of time such as weeks, months.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

COCOMO

- **Software are categorized into :**

1. **Organic:** if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.

2. **Embedded Projects:** A software project with requiring the highest level of complexity, creativity, and experience requirement fall under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models.

3. **Semi-detached projects:** if the vital characteristics such as team-size, experience, knowledge of the various programming environment lie in between that of organic and Embedded.

The projects classified as Semi-Detached are comparatively less familiar and difficult to develop compared to the organic ones and require more experience and better guidance and creativity. Eg: Compilers or different Embedded Systems can be considered of Semi-Detached type.

COCOMO

Coefficients Related to Basic Model

Estimation at the early design stage where ONLY User Requirements are defined

$$\text{Effort} = a * (\text{KLOC})^b$$

$$\text{Project Duration} = c * (\text{Effort})^d \quad (\text{represented in Months})$$

$$\text{Person Required} = \text{Effort} / \text{Project Duration}$$

Development Model	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Embedded	3.6	1.20	2.5	0.32
Semi detached	3.0	1.12	2.5	0.35



COCOMO

Suppose a project manager has used FP analysis to estimate lines of source code required to develop a project and finds that the code size is 10,000. Estimate duration of the project and number of persons required.

- a) Assume the project is classified as **Embedded Project**.
- b) Assume the project is classified as **Organic Project**.

Development Model	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Embedded	3.6	1.20	2.5	0.32
Semi detached	3.0	1.12	2.5	0.35

$$\text{Effort} = a * (\text{KLOC})^b$$

$$\text{Duration} = c * (\text{Effort})^d$$

$$\text{Person Required} = \frac{\text{Effort}}{\text{Duration}}$$

a) Effort = 57.06 , Project duration = 9.12 months and person = 6.26 i.e. 6

b) Effort = 26.92 , Project duration = 8.72 months and person = 3.08 i.e. 3

COCOMO

- Suppose a project was estimated to be 400 KLOC. Calculate the effort and development time for each of the three model i.e., organic, semi-detached & embedded ,persons required

(i)Organic Mode

$$E = 2.4 * (400)1.05 = 1295.31 \text{ PM}$$
$$D = 2.5 * (1295.31)0.38 = 38.07 \text{ PM}$$
$$P = ?$$

(ii)Semidetached Mode

$$E = 3.0 * (400)1.12 = 2462.79 \text{ PM}$$
$$D = 2.5 * (2462.79)0.35 = 38.45 \text{ PM}$$

(iii) Embedded Mode

$$E = 3.6 * (400)1.20 = 4772.81 \text{ PM}$$
$$D = 2.5 * (4772.8)0.32 = 38 \text{ PM}$$

- A project size of 200 KLOC is to be developed. Software development team has average experience on similar type of projects. The project schedule is not very tight. Calculate the Effort, development time, average staff size, and productivity of the project.

The semidetached mode is the most appropriate mode, keeping in view the size, schedule and experience of development time.

Hence $E = 3.0(200)^{1.12} = 1133.12 \text{ PM}$
 $D = 2.5(1133.12)^{0.35} = 29.3 \text{ PM}$

$$\begin{aligned} \text{Effort} &= a * (\text{KLOC})^b \\ \text{Duration} &= c * (\text{Effort})^d \\ \text{Person Required} &= \frac{\text{Effort}}{\text{Duration}} \end{aligned}$$

$$\begin{aligned} \text{Average Staff Size (SS)} &= \frac{E}{D} \text{ Persons} \\ &= \frac{1133.12}{29.3} = 38.67 \text{ Persons} \end{aligned}$$

$$\text{Productivity} = \frac{\text{KLOC}}{E} = \frac{200}{1133.12} = 0.1765 \text{ KLOC/PM}$$

$$P = 176 \text{ LOC/PM}$$

COCOMO-II model

- COCOMO-II is the revised version of the original Cocomo (Constructive Cost Model) and is developed at University of Southern California. It is the model that allows one to estimate the cost, effort and schedule when planning a new software development activity.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



COCOMO II is actually a hierarchy of estimation models that address the following areas:

- **Application composition model.** Used during the early stages of software engineering, when prototyping of user interfaces, consideration of software and system interaction, assessment of performance, and evaluation of technology maturity are paramount.
- **Early design stage model.** Used once requirements have been stabilized and basic software architecture has been established.
- **Post-architecture-stage model.** Used during the construction of the software.

COCOMO II

- COCOMO II models require sizing information. Three different sizing options are available as part of the model hierarchy: **object points, function points, and lines of source code.**
- The object point is an indirect software measure that is computed using counts of the number of
 - (1) screens (at the user interface),
 - (2) reports, and
 - (3) components likely to be required to build the application.

Each object instance (e.g., a screen or report) is classified into one of three complexity levels (i.e., simple, medium, or difficult)

COCOMO-II

- complexity is a function of the number and source of the client and server data tables that are required to generate the screen or report and the number of views or sections presented as part of the screen or report

Object type	Complexity weight		
	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	8
3GL component			10



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



COCOMO-II

- When component-based development or general software reuse is to be applied, the percent of reuse (%reuse) is estimated and the object point count is adjusted:

$$\text{NOP} = (\text{object points}) \times [(100 - \% \text{reuse}) / 100]$$

where NOP is defined as new object points

- To derive an estimate of effort based on the computed NOP value, a “productivity rate” must be derived.

$$\text{PROD} = \frac{\text{NOP}}{\text{person-month}}$$



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



COCOMO-II

- Project efforts

$$\text{Estimated effort} = \frac{\text{NOP}}{\text{PROD}}$$



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



COCOMO-II

FIGURE 26.7 Productivity rate for object points.

Source: [Boe96].

Developer's experience/capability	Very low	Low	Nominal	High	Very high
Environment maturity/capability	Very low	Low	Nominal	High	Very high
PROD	4	7	13	25	50



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



COCOMO – II Example

• Use the COCOMO II model to estimate the effort required to build software for a simple ATM that produces 12 screens, 10 reports, and will require approximately 80% as new software components. Assume average complexity and average developer/ environment maturity. Use the application composition model with object points.

• Given

Object	Count	Complexity	Weight Factor	Total Objects
Screen	12	Simple	1	12
Report	10	Simple	2	20
3GL Components	0	NA	NA	0
Total Objects Points :				32



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

- It is given that 80% of components have to be newly developed. So remaining 20% can be reused

- Now compute new object points as

$$\text{NOP} = (\text{object points}) * [(100 - \% \text{reuse}) / 100]$$

$$\text{NOP} = 32 * (100 - 20) / 100 = 32 * 80 / 100$$

$$\text{NOP} = 25.6 \text{ object points}$$

- Since productivity is given average, we can assume PROD = 13

- Hence, $\text{effort} = \text{NOP} / \text{PROD}$

$$\text{effort} = 25.6 / 13$$

$$\text{effort} = 1.96 \text{ person months}$$



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



1.4

- 1.4 Project Management activities :
Planning, Scheduling and Tracking



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Planning & Scheduling

What will be the inputs for planning ?

Scope (What to develop)

Estimation (Time and hence cost)

Risks associated

Human Resources available (role, number, when will be available etc.)



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Planning & Scheduling

Steps in Planning

- Prepare a task table mentioning the task name, task id, duration, resource name
- Create a network of these tasks that will enable to get the job done on time

Some observations:

- In a complex system many tasks may occur in parallel
- Result of work performed during one task may have a profound effect on work to be conducted in another task
- Interdependencies are very difficult to understand without a schedule
- Virtually impossible to assess progress on a moderate or large software project without a detailed schedule.

Reasons for late delivery of products

- An **unrealistic deadline** established by someone outside the software development group and forced on managers and practitioners within the group.
- **Changing customer requirements** that are not reflected in schedule changes.
- **Predictable and/or unpredictable risks** that were not considered when the project commenced.
- **Technical difficulties** that could not have been foreseen in advance.
- **Human difficulties** that could not have been foreseen in advance.
- **Miscommunication** among project staff that results in delays.
- A **failure by project management** to recognize that the project is falling behind schedule and a lack of action to correct the problem.

Late delivery

Remedies:

- Perform a detailed estimate using historical data from past projects. Determine the estimated effort and duration for the project.
- Using an incremental process model, develop a strategy that will deliver critical functionality by the imposed deadline, but delay other functionality until later. Document the plan.
- Avoid to impose unrealistic deadline

Project Scheduling

- An activity that distributes estimated effort across the planned project duration by allocating the effort to specific tasks
- specific software tasks are identified and scheduled
- the schedule evolves over time
- During early stages of project planning, a macroscopic schedule is developed identifying all major activities and the product functions to which they are applied.
- As the project gets under way, refine into a detailed schedule.

Two approaches:

- Based on the end-date for release distribute effort within the prescribed time frame
- Based on the time required for each activity, decide the end date

Basic principle guides of Scheduling

- **Compartmentalization**: Break into a number of manageable activities and tasks by decomposing both the product and the process
- **Interdependency**: The interdependency of each compartmentalized activity or task must be determined. Some tasks must occur in sequence while others can occur in parallel. Some activities cannot commence until the work product produced by another is available. Other activities can occur independently.
- **Time allocation** Each task to be scheduled must be allocated some number of work units (e.g., person-days of effort). In addition, each task must be assigned a start date and a completion date that are a function of the interdependencies and whether work will be conducted on a full-time or part-time basis.
- **Effort validation**. Every project has a defined number of staff members. As time allocation occurs, the project manager must ensure that no more than the allocated number of people have been scheduled at any given time.

Basic principle guides of Scheduling

- **Define responsibilities** Every task that is scheduled should be assigned to a specific team member.
- **Define outcomes** Every task that is scheduled should have a defined outcome. For software projects, the outcome is normally a work product (e.g., the design of a module) or a part of a work product. Work products are often combined in deliverables.
- **Define milestones** Every task or group of tasks should be associated with a project milestone. A milestone is accomplished when one or more work products has been reviewed for quality and has been approved.

Some facts about Scheduling

- Experienced resources would need less time for development
- Familiar tools & defined process will reduce the development efforts
- **Adding more programmers/ developers will NOT ALWAYS reduce the project duration**
- A recommended distribution (not rules) of effort across the definition and development phases is often referred to as the 40–20–40 rule (Forty percent of all effort is allocated to front-end analysis and design Twenty percent for coding and Forty percent applied to back-end testing)

Defining Task set

- Irrespective of the process model used for development, task set that enable a software team to define, develop, and ultimately support computer software.
- A task set is a collection of software engineering work tasks, milestones, and deliverables that must be accomplished to complete a particular project.
- The task set to be chosen must provide enough discipline to achieve high software quality.
- **It must NOT burden the project team with unnecessary work.**
- In order to develop a project schedule, a task set must be distributed on the project timeline.

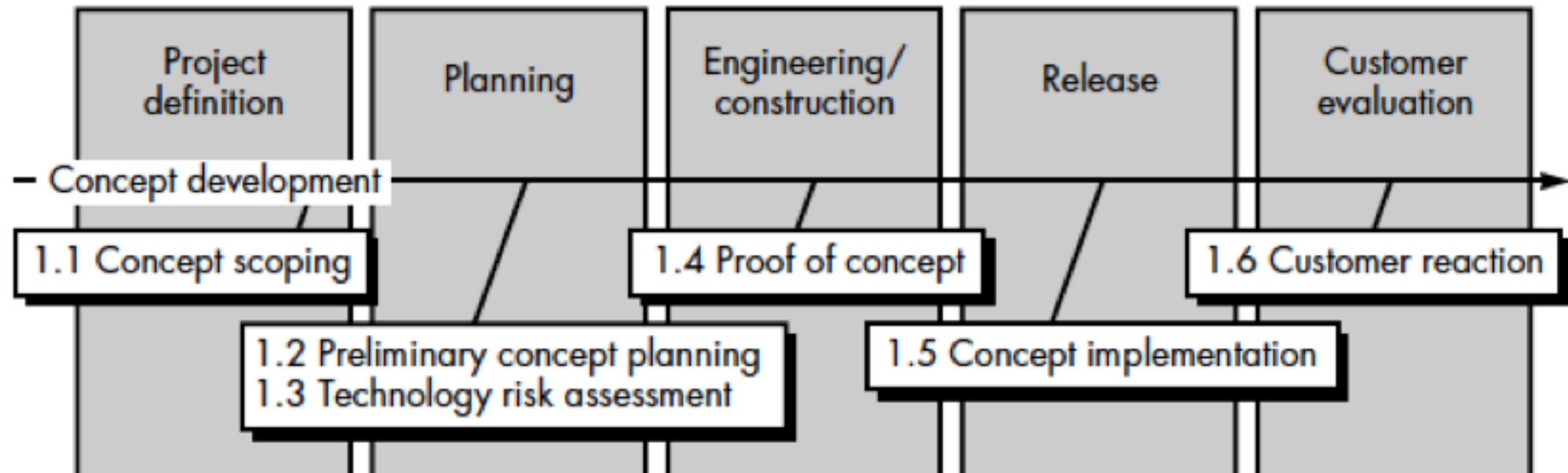
Defining Task set

- **Based on phase of the project identify the various major tasks**

Concept , Analysis , Design, Coding, Maintenance

- **Concept scoping** determines the overall scope of the project.
- **Preliminary concept planning** establishes the organization's ability to undertake the work implied by the project scope.
- **Technology risk assessment** evaluates the risk associated with the technology to be implemented as part of project scope.
- **Proof of concept** demonstrates the viability of a new technology in the software context.
- **Concept implementation** implements the concept representation in a manner that can be reviewed by a customer and is used for “marketing” purposes when a concept must be sold to other customers or management.
- **Customer reaction** to the concept solicits feedback on a new technology concept and targets specific customer applications.

Sample Linear Development Approach Tasks



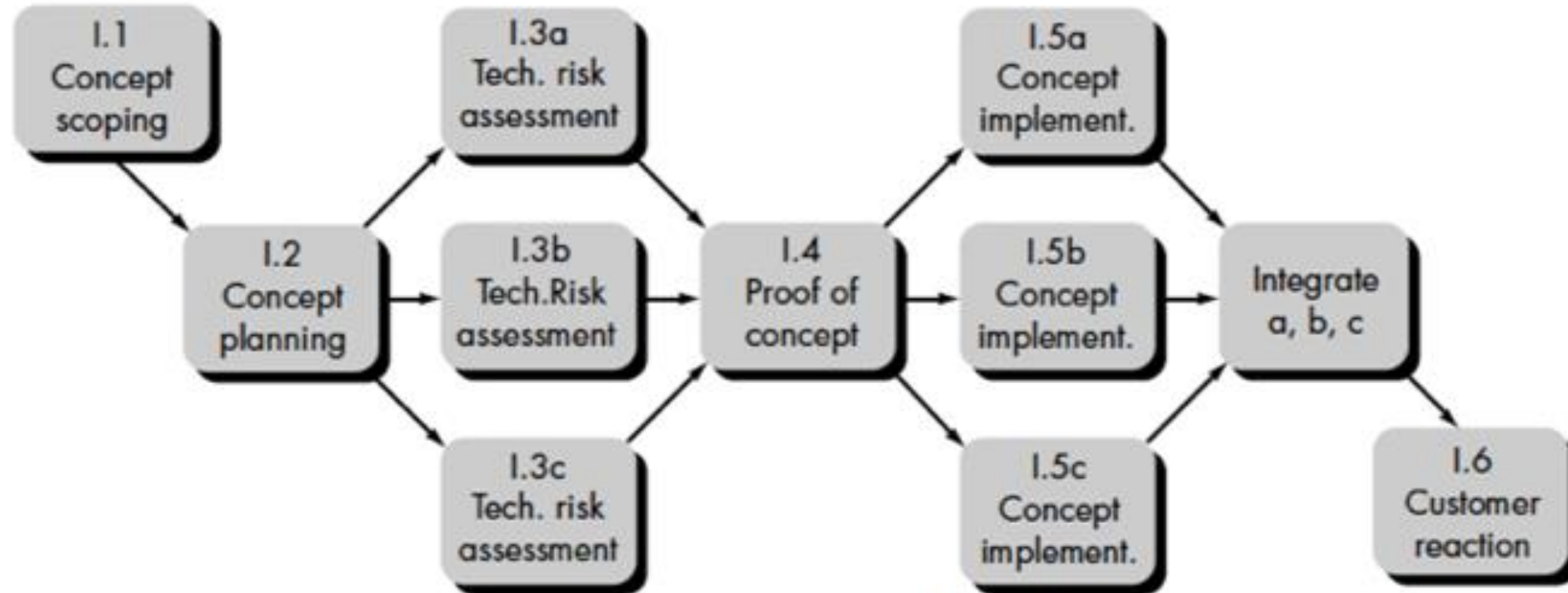
SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Sample Task Table

ACTIVITY	DESCRIPTION	IMMEDIATE PREDECESSORS	DURATION (WEEKS)
H	Basic design	—	10
I	Hardware design for A	H	8
J	Hardware design for B	H	6
K	Drawings for B	J	4
L	Software specifications	J	2
M	Parts purchase for B	J	4
N	Parts purchase for A	I	4
O	Drawings for A	I	5
P	Installation drawings	I, J	5
Q	Software purchases	L	5
R	Delivery of parts for B	M	5
S	Delivery of parts for A	N	3
T	Software delivery	Q	3
U	Assembly of A	O, S	1
V	Assembly of B	K, R	5
W	Test A	U	2
X	Test B	V	3
Y	Final installation	P, W, X	8
Z	Final system test	Y, T	6

Sample Task Network Diagram



Three I.5 tasks are applied in parallel to 3 different concept functions

Scheduling

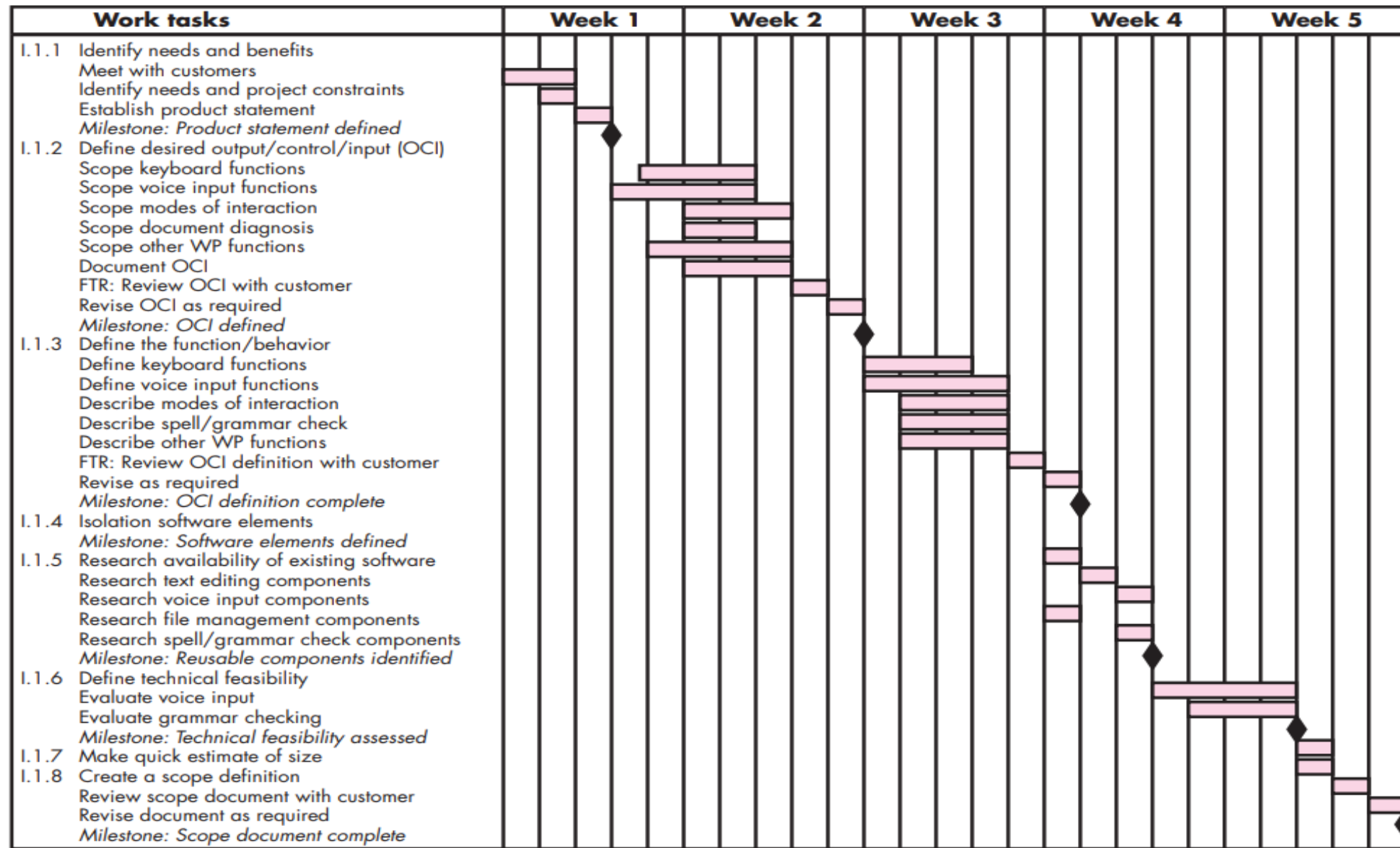
Project Scheduling methods:

- Critical Path Method (CPM)
- Program Evaluation & Review Technique (PERT)

Tools:

- Timeline Chart/ Gantt Chart
- MicroSoft Project Tool used for scheduling

Scheduling



Tracking the Schedule

- The project schedule provides a road map
- Properly developed project schedule defines the tasks and milestones that must be tracked and controlled as the project proceeds
- Tracking can be accomplished by:
 - Conducting periodic project status meetings in which each team member reports progress and problems.
 - Evaluating the results of all reviews conducted throughout the software engineering process.
 - Determining whether formal project milestones (the diamonds shown in Figure) have been accomplished by the scheduled date
 - Comparing actual start-date to planned start-date for each project task listed in the resource table
 - Meeting informally with practitioners to obtain their subjective assessment of progress to date and problems on the horizon.
 - Using earned value analysis to assess progress quantitatively.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering